



SYNAPTIC OVERPRODUC- TION

Factorio 2.0.77

Modlist & Guide

Version 2.0.77 - 2026

Table of Contents

Contents

Table of Contents	2
Installation Guide	4
Game Prep	4
Enable the Built-In Mod Manager	5
Fetch the Mods	5
Version Compatibility	5
The Wave 2 Anchor: Krastorio 2 and the Spaced Out Bridge	6
Launch & Verify	6
Per-Wave Switching	7
Waves & New-Game Setups	8
Wave 0: The Castaway	8
The Castaway: How to Play	8
The Castaway's Roadmap	8
Controls & Keybindings	8
The Interface Tour	10
Your First Minutes	10
Power & the Electric Grid	12
Automation Basics	12
Science Packs & Research	13
Oil Processing	14
Early Rail	15
Biters & Defense	16
Launching the Rocket	17
Common Beginner Mistakes	18
The Castaway's First Session	18
The Castaway: Modlist	19
The Castaway: Mechanics	19
The Castaway: Graphics	21
Wave 1: The Architect	22
The Architect: Strategy	22
The Bus: Order Out of Spaghetti	22
Trains: Signals, Blocks, and a Network That Scales	23
Circuits: The Factory's Nervous System	23
Logistics Robots: When Bots Beat Belts	24
Modules & Beacons: Spending Spare Throughput	25
Quality: Better Things, Not Just More	26
Rate Mathematics: Counting the Factory	26
Megabase Blocks: The Architect's Capstone	27
The Architect's Roadmap	28
The Architect: Modlist	28

The Architect: Mechanics	29
The Architect: Content	30
The Architect: Graphics	32
Wave 2: The Krastorio Prodigy	35
The Krastorio Prodigy: Strategy	35
The New Ores: Imersite, Mineral Water, Rare Metals	35
New Intermediates & Processing Chains	36
Research Overhaul: Tech Cards & Research Computers	36
Adapting the Vanilla Systems	37
Combat & Biters: Ballistics Against Snappers	38
The Endgame: Deep Science and the Win	39
The Prodigy's Arc	39
The Krastorio Prodigy's Roadmap	40
The Krastorio Prodigy: Modlist	41
The Krastorio Prodigy: Mechanics	41
The Krastorio Prodigy: Content	42
The Krastorio Prodigy: Graphics	45
Glossary	47

Installation Guide

This chapter takes you from a fresh Factorio install to a running, verified modded setup: the exact game build, the built-in mod manager, where every mod in this guide comes from, how version compatibility is enforced, and the per-wave discipline the whole guide assumes. It is written against **Factorio 2.0.77**, released **May 21, 2026**, with the **Space Age** expansion owned. The mod organizer is Factorio's own built-in mod manager — the **Mods** screen in the main menu — so there is no third-party tool to install and nothing to configure outside the game. Every mod in the list is published on the official Factorio Mod Portal (mods.factorio.com), and every card in the wave chapters records the portal-verified build that was checked against this install.

If you follow the waves in order, you do the setup here exactly once, up front. Each wave is its own new game with its own mod subset; the final section of this chapter explains how to switch between waves using only the built-in manager's checkboxes.

The whole chapter, in one glance:

1. **Game prep** — Factorio 2.0.77 from Steam or standalone, confirmed.
2. **Enable the built-in manager** — main menu → Mods, synced to the official portal.
3. **Fetch the mods** — per-card portal URLs, installed through the manager.
4. **Version compatibility** — how the manager guards against mismatched builds.
5. **The Wave 2 anchor** — Krastorio 2 and the Spaced Out bridge.
6. **Launch & verify** — the in-game mod state is the ground truth.
7. **Per-wave switching** — a new game per wave, checkboxes for the rest.

Each step is one short section below. None of them should take more than a few minutes, and the order they are written in is the order you should do them — fetching the mods depends on the manager being open, which depends on the game being prepped.

Game Prep

Install Factorio from Steam, or download the standalone build from factorio.com — both give you the same game, and the rest of this chapter is identical either way. Run the game once before you touch anything else: that first launch generates the game's install folders and confirms the build in front of you.

The build matters. This guide targets **2.0.77**, released **May 21, 2026** — the exact build every mod card was verified against and every fact in the wave chapters was checked against. The main menu shows the version; confirm it reads 2.0.77 before you begin. If you install through Steam and the game later updates to a newer build, re-confirm the version before starting a wave — the guide's guarantees are written for 2.0.77 specifically.

Note: The 2.0 version line is shared between the base game and the Space Age expansion — there are not separate “2.0.77 base” and “2.0.77 expansion” builds. The same version number covers both, and a mod declaring `factorio_version` “2.0” is compatible with the whole 2.0 line. What differs between waves is not the build but which **mods** are enabled — which is the whole subject of this chapter.

Space Age is owned and installed for this guide, and it behaves like a mod: it appears in the mod manager as an enabled or disabled entry. The guide uses it selectively — the Wave 2 chapter runs **with** Space Age enabled, while Waves 0–1 run the base-game experience. The Wave 2 anchor section below explains the difference and why it exists.

Enable the Built-In Mod Manager

Factorio ships its mod manager inside the game: main menu → **Mods**. There is no separate application to install, no account to configure, and no launcher step — the manager is part of the game itself, and it is the only tool this guide needs.

The manager is a portal client. It syncs with the official Factorio Mod Portal (mods.factorio.com), so the catalogue you browse in the game is the same catalogue every mod card in this guide links to. You can browse by category, search by name, and install any mod with one click; the manager downloads the mod into the game's `mods/` folder, adds it to your list, and enables it for the next start.

Two built-in behaviours do the work this guide leans on, and both are automatic:

- **Dependency auto-resolution.** When a mod declares dependencies, the manager installs them too. This is how **Krastorio 2** pulls in its supporting asset and menu-simulation mods, and how **Krastorio 2 Spaced Out** pulls in the bridge's own assets — you never fetch those by hand.
- **Version-aware flags.** A mod whose releases do not match your build is flagged rather than silently installed, which is the guard the Version Compatibility section relies on.

The manager's list is also where every wave's on/off decisions happen: each installed mod is a checkbox, enabled mods load on the next start, and disabled mods stay installed but idle. That checkbox list is the installation state for the whole guide.

Fetch the Mods

Every mod in this guide lives on the official portal, and every card in the wave chapters carries its verified URL — the same page the manager searches. Two examples: **Krastorio 2** and **Even Distribution**. The per-card URL is the canonical source: if you ever doubt whether you have the right mod, the card's link settles it.

To fetch a wave's mods:

1. Open the **Mods** screen from the main menu.
2. Search for each mod by name — the manager's search reaches the same portal catalogue the cards point at.
3. Hit **Install** and let the manager place the mod in the game's `mods/` folder.
4. Watch the dependency note on install — anything the manager pulls in as a dependency is expected, not an error.

Fetch each wave's mods when you reach that wave, not all at once. The mods are listed per wave in that wave's Modlist section, and the Per-Wave Switching section explains why installing everything up front would fight the guide's design.

Version Compatibility

The portal lists every release of a mod with its `factorio_version` — the version line that release is built for. When your game's build and the mod's `factorio_version` do not match, the manager flags the mod as incompatible rather than letting it silently break the game. That is the manager's compatibility guard, and it is the reason every card in the wave chapters records a build: the "Factorio version" column on each card states the portal-verified 2.0-line release the manager will select for a 2.0.77 install, and the STATUS.md audit trail logs the VERIFIED verdict behind every one of them.

Because this guide targets the 2.0 line, every accepted mod was required to have a 2.0-line release — a mod whose newest build targets Factorio 1.1 or 2.1 is outside this install and was rejected or deferred during the wave verification. A correct install should therefore produce no compatibility flags at all: every mod in the list is verified to match a 2.0.77 install, and if the manager flags one of them, the card's recorded build is the evidence to check against.

Note: A naming trap worth knowing: a mod’s page shows its newest release on **any** version line, which is not necessarily the one your build wants. Krastorio 2 Spaced Out’s page displays “2.0.14” as its latest version, but that is the 2.1-line build; the 2.0 line uses 1.6.x version numbers. For a 2.0.77 install the manager selects the 2.0-line build — 1.6.21 — and that is the build the card records. The same principle applies to any mod whose page headline is newer than its 2.0-line build.

The Wave 2 Anchor: Krastorio 2 and the Spaced Out Bridge

One wave breaks the pattern, and it owns the guide’s most important single instruction: **keep Space Age enabled for Wave 2.**

Wave 2 is anchored by **Krastorio 2** (mods.factorio.com/mod/Krastorio2), the end-game overhaul. On its own, though, Krastorio 2 does not work with Space Age. Its page is explicit: “Krastorio 2 does not properly support the Space Age mod. The game will load, but progression will be broken. Please install an add-on such as Krastorio 2 Spaced Out.”

The add-on is **Krastorio 2 Spaced Out** (K2SO, mods.factorio.com/mod/Krastorio2-spaced-out) by Polka_37, which merges Krastorio 2 with Space Age into one longer playthrough. K2SO **requires** Space Age — its releases carry the space-travel feature flag — so Wave 2’s install is: Space Age enabled, Krastorio 2, Krastorio 2 Spaced Out, and the wave’s seven verified companions. On a 2.0.77 install the manager selects each mod’s 2.0-line build — Krastorio 2 **2.0.19** and K2SO **1.6.21** — both logged as VERIFIED - COMPATIBLE in STATUS.md.

Warning: The honest limit of the bridge: K2SO’s page states it “won’t receive updates or bugfixes for Factorio 2.0, switching to Factorio 2.1 is preferable.” The 2.0 line is stable and verified at 1.6.21, which is exactly what this guide targets — but if you ever move the game to a newer build, re-confirm the bridge before carrying a save. Waves 0–1 never touch this: their mods are base-game-compatible, and their chapters describe the base game’s rocket rules, not Space Age’s.

Launch & Verify

With a wave’s mods installed and its checkboxes set, start the game. Then verify the load actually worked:

1. From the main menu, open the **Mods** screen. It lists every installed mod and which ones are enabled — this is the authoritative state, not a third-party view.
2. Confirm every mod that wave’s Modlist section names is present and enabled, and that nothing you expected is missing.
3. Confirm nothing is flagged incompatible. A flag here means the manager refused the mod for your build — see Version Compatibility for what to check.
4. Start a **new game** for the wave and play the first minutes of its opening to confirm the mods behave before committing to the run. A wave’s mechanics mods (new ores, new science, logistics dispatch) are best confirmed by actually using them, not just by checking the list.

Tip: The in-game mod state is the ground truth — it is exactly what the game will load, no further translation needed. If a mod you enabled is not showing its effects in play, the first check is the Mods screen: is it really enabled, and is the running save the new-game start that wave created?

Per-Wave Switching

The three waves are three separate games, not three chapters of one save. Each wave is its own new map with its own mod subset, and the mods are layered so that the **Wave 0 base carries into every wave** while each wave's content stays scoped to that wave:

- **Wave 0 — The Castaway:** the base game plus the seven-mod QoL base — Even Distribution, Squeak Through 2, Rate Calculator, Factory Planner, YARM, Todo List and Bottleneck Lite. This is the persistent core of all three waves.
- **Wave 1 — The Architect:** the Wave 0 base plus the twelve Wave 1 mods — Miniloader (Redux), Deadlock's Stacking Beltboxes & Compact Loaders, Warehousing Mod, Project Cybersyn, Circuit HUD V2, Module Inserter, Auto Deconstruct, Nixie Tubes, Belt Visualizer, Prismatic Belts, Circuit Visualizer and Alien Biomes. Runs the base game (Space Age off).
- **Wave 2 — The Krastorio Prodigy:** the Wave 0 base plus the two anchors — Krastorio 2 and Krastorio 2 Spaced Out — plus the seven verified companions: Krastorio 2 Imersite Asteroids, Armoured Biters, FNEI, Fluid Must Flow, Milestones, K2 Tech Card Icons for Space Age, and Krastorio2 - roboport mode colored textures. Runs **with** Space Age on.

The Wave 0 QoL base persists across all three because it cannot touch gameplay data — pure client, UI and informational mods carry unchanged into Krastorio 2's rebalanced world. The Wave 1 content mods deliberately do **not** carry into Wave 2, because Krastorio 2 rebalances every recipe and the wave keeps its install focused on the two overhauls and their companions. Space Age is disabled for Waves 0–1 (their content is the base-game experience — the base game's rocket rules, not Space Age's) and enabled for Wave 2, where the K2SO bridge requires it.

Switching waves is two steps in the Mods screen:

1. **Enable the new wave's subset and disable the previous wave's content** — every installed mod is a checkbox, so moving from Wave 1 to Wave 2 means checking the K2 pair and its companions and unchecking the Wave 1 content. The Wave 0 base stays checked throughout.
2. **Start a new game for the wave.** Each wave gets a fresh map — which is also where worldgen mods like Alien Biomes do their work, because an existing save never sees a new terrain mod.

Tip: Before committing to a long run, glance at the Mods screen and read the enabled list against that wave's Modlist section. The enabled list **is** the wave — it is the whole definition of what you are about to play.

Warning: Two mistakes break a wave, and both are checkbox errors. Carrying Wave 1's recipe-adjacent content into Wave 2 fights Krastorio 2's rebalanced recipes. And starting Wave 2 with Space Age disabled — or with Krastorio 2 enabled but K2SO missing — reproduces exactly the broken progression Krastorio 2's own page warns about. The Mods screen is where both errors live and where both are fixed.

Waves & New-Game Setups

Wave 0: The Castaway

The Castaway: How to Play

The survey ship **Nightjar** came apart over Nauvis and you are what survived: a burner drill, a stone furnace, a burner inserter, a wooden chest, and the engineer's instinct to bootstrap a factory from raw ore before the biters notice you. This chapter teaches everything a castaway needs, in the order you will actually need it: how to control the game and read the interface, how to survive your first minutes, how to automate the first machine lines, and then the whole bootstrap campaign — science, oil, rail, defense — that ends with the first rocket leaving the planet.

Everything here describes vanilla Factorio 2.0.77. Wave 0 installs no content mods; the seven mods in the Modlist chapter only make the game easier to see, easier to operate, and easier to plan.

The Castaway's Roadmap

Before the keybindings and the machinery, here is the whole wave in one glance — six goals that take you from crash site to rocket, each one building on the last. You do not need to follow them exactly; they exist so you always know what “progress” means next.

1. **Survive the first hour.** Mine by hand, build the burner loop, and reach electricity — one offshore pump, one boiler, one steam engine, and electric drills on coal and iron. This is **Your First Minutes**.
2. **Automate iron.** Belts, inserters and an assembling machine turn the smelting loop into a machine line that runs while you build something else. This is **Automation Basics**.
3. **Research red and green science.** Two assembler lines feeding labs unlock belts, inserters and automation 2 — the tools you will scale with. This is **Science Packs & Research**.
4. **Face oil.** Pumpjacks, a refinery and cracking deliver plastic and sulfur, unlocking chemical science and the whole mid-game economy. This is **Oil Processing**.
5. **Bring the ore to the base.** When the starter patches thin out, a train line hauls coal and iron from afar instead of a thousand belts. This is **Early Rail**.
6. **Hold the line, then fly.** Walls and turrets buy the peace you need to build the rocket silo and its three ingredient belts, and the satellite carries you home. This is **Biters & Defense** and **Launching the Rocket**.

Each goal is small enough to finish in one sitting, and each one leaves you with a machine that keeps producing while you move to the next. That is the whole trick of Factorio — the factory never sleeps, so neither does progress.

Controls & Keybindings

Factorio is a top-down factory simulator: you walk an engineer around, build machines, and the machines do the work. Almost everything has a keyboard default, and every one of them can be remapped under **Settings** → **Controls**. The defaults below are what 2.0.77 ships with.

Tip: Alt-mode is the most important toggle in the game. Press **Alt** once and every machine, chest and belt shows what it contains or produces on top of its sprite. New players who skip it are flying blind — press Alt before you do anything else, and never turn it off. In short, it shows every entity's recipe, contents and direction at a glance.

Category	Action	Input
Movement	Move the engineer	WASD

Camera	Zoom in / out	Mouse wheel
Camera	Rotate the camera	Hold Shift + mouse wheel
Mining	Mine a tile / entity	Hold Left mouse (or F)
Build	Place the selected item	Left mouse
Select	Pipette an entity (pick its item)	Q
Build	Rotate / reverse-rotate a ghost	R / Shift + R
Build	Copy an entity's settings	Shift + Right mouse
Build	Paste settings onto entities	Shift + Left mouse
Build	Place a ghost / blueprint	Shift + Left mouse
Build	Blueprint library	B
Build	Undo / redo last action	Ctrl + Z / Ctrl + Y
Inventory	Transfer item stack quickly	Ctrl + Left mouse
Inventory	Split a stack in half	Ctrl + Right mouse
Inventory	Set an inventory slot filter	Middle mouse
Interface	Open inventory & crafting	E
Interface	Show info overlay (alt-mode)	Alt
Interface	Toggle map	M
Interface	Production statistics	P
Interface	Logistics network view	N
Interface	Quickbar slots	1–0
Interface	Cycle quickbar pages	~ key (the key above Tab)
Interface	Debug / FPS overlays	F4 / F5 / F6 / F7

Tip: The mouse does most of the work. **Left-click** places what is in your cursor, **right-click** (with nothing selected) deconstructs a single entity, and holding **Ctrl** while clicking makes the stack transfer fast. When you are lost, press **Q** while hovering a machine — it puts that machine's item in your hand so you can build more of it without hunting through menus.

Warning: Don't fight the copy-and-paste system. To replicate one assembler's settings across a row, **Shift + right-click** the original, then **Shift + left-click** (or drag across) the row — recipes, requesters and filters all copy. Hand-rebuilding ten identical machines instead of copying is the most common early time-waster.

The build tools. Beyond placing entities directly, three tools do the planner's work and all live in the shortcuts bar or the toolbar:

- **Ghosts** — place a building you do not yet have the materials for (Shift + left-click) and it appears as a translucent outline that gets filled in later, either by you or by construction robots once you reach Wave 1. Ghosting a layout first means the factory is planned before it is paid for.
- **Deconstruction planner** — pick it up and drag it over anything to mark it for removal; deconstructing returns the materials, and in 2.0 the ghosts and markers can be placed straight from the map view too.
- **Upgrade planner** — drag it over a row of yellow belts and it replaces them with red belts (if you have them) in one gesture. This is how an early factory scales without being torn down.

Tip: The deconstruction planner doubles as the eraser for your own mistakes. Mark a mislaid line, walk away, and the engineers who return to a tidy site are not the ones who panic — deconstruction in Factorio is free and full-value, so there is never a reason to live with a layout you dislike.

The Interface Tour

The whole HUD is built around a few panels, and once you know what each one tells you, the game opens up.

- **Toolbar / quickbar (bottom):** the hotbar of building items, split into pages. Pick a slot with the number keys or the ~ key to switch pages; whatever is in your hand, you place with Left mouse. The shortcuts bar at the far right holds the blueprints, deconstruction planner, and special tools.
- **Crafting menu (E):** the inventory and the hand-crafting list side by side. Anything you can currently craft is shown; click a recipe to craft one, **Shift + click** to queue five. The queue at the top-left keeps the list of things you are making.
- **Minimap (top-right):** a live satellite view of the world around you. Press **M** for the full map, where you can zoom, plot with ghost markers, and see radar range. Build a radar early — it reveals the map around your base so you can plan instead of guess.
- **Production statistics (P):** graphs of every item, fluid and energy source over time. When something is slowing down, P shows you exactly which line stopped producing — the castaway's best diagnosis tool.
- **Logistics network view (N):** the status of your logistic network — which chests are in it, what they hold, how many robots are working. Nothing uses this at Wave 0's start; it matters once construction and logistics robots arrive.
- **Alt-mode overlay (Alt):** recipes on assemblers, contents on chests, arrows on inserters. Keep it on. Always.
- **Tooltips:** hover anything — an entity, a recipe, a research card — and a tooltip explains what it is and what it needs. The game's entire documentation lives in these tooltips, and the wiki is the reference companion.

Reading the factory. Once the panels are familiar, learn to read them as a diagnosis. The **Production statistics** screen (P) graphs every item, fluid and energy source over time: a flat iron-plate line with hungry consumers downstream is a smelting bottleneck; a climbing power line that suddenly stops is a dead coal feed. The **Kills** tab shows how many biters you have cleared, which matters because killing spawners raises evolution. And the **alert icons** that flash along the top of the screen when something is under attack, out of ammo, or out of power are the factory telling you its problems in real time — a factory that alerts you about its own failures is a factory you can actually manage.

Tip: The minimap and the full map are your planning canvas. Press **M**, then right-click to drop a marker on the map — a reminder of where the iron patch you promised yourself actually is. Markers survive sessions and are the cheapest project tracker the game gives you (the **Todo List** mod in the Modlist chapter adds the human-readable version).

Your First Minutes

Your starting kit — burner mining drill, stone furnace, burner inserter, wooden chest — is enough to leave the crash site and make the first iron that the whole factory will be built from. The loop you are about to build is the loop that never stops: **mine ore, burn fuel to smelt it, turn the plates into machines.**

Mining by hand. Walk to a stone, iron or coal patch — the black-ish rocks are coal, the brown ones are iron — hold Left mouse on a tile and the engineer chips it out. Everything you pick up lands in your inventory. Hand-mining is slow on purpose: it exists to motivate the machine that does it for you.

The starter burner loop. Place the **burner mining drill** on an iron or coal patch. A burner drill feeds itself: give it a little fuel once (any flammable item), and it will keep mining and eat from its own output while it runs. Put the **stone furnace** right where the drill's output tile lands, put coal in the furnace's fuel slot, and iron ore fed from the drill will smelt into iron plates automatically. The

burner inserter and **wooden chest** complete the scene — the inserter pulls coal off the drill and pushes it into the furnace, and the chest holds overflow.

Warning: Burner machines are self-contained but hungry: a burner drill stops the moment its own fuel runs out, and if you let it eat the coal it is mining you get no coal at all. In the first minutes, mine a coal patch by hand into a separate stack before you trust the drill to fuel itself. The wiki notes the same caution applies to boilers — “if no power is being produced, then powered inserters fueling a boiler cannot operate”, which is why the first boiler should be fed by a burner inserter.

Coal, furnace, steam power. Before you build anything electrical you need fuel and power. The burner drill mines coal; the furnace burns it. The jump to electricity is the classic **boiler** → **steam engine** line: an **offshore pump** on a lake edge, one **boiler** beside it, and a **steam engine** beside the boiler. Boilers turn water into 165°C steam (a boiler produces 60 steam per second and burns a piece of coal roughly every 2.2 seconds); the steam engine turns it into electricity (900 kW per engine, 30 steam per second). The 2.0.7 ratio is two steam engines per boiler, and one offshore pump feeds up to 200 boilers. A single pump → 200 boilers → 400 engines is roughly 360 MW — far more than you will need for many hours.

The first ten minutes, step by step. If you want a concrete script rather than theory, here is one that works on almost any map:

1. Walk to the nearest iron patch and hand-mine a few stacks of iron ore while you scout for coal and a lake. Keep the coal patch in sight.
2. Place your burner mining drill on the coal patch and feed it one piece of fuel by hand — it now mines coal forever on its own.
3. Place the stone furnace on the coal drill’s output tile, and the burner inserter between the coal drill and the furnace so coal flows into the furnace automatically. Drop iron ore into the furnace’s input slot and it smelts while you work.
4. Hand-craft and place an **offshore pump** on the lake edge, a **boiler** right beside it, and a **steam engine** beside the boiler. Feed the boiler by hand once with coal (or let a burner inserter do it) — you now have your first electricity.
5. Craft **electric mining drills**, **assembling machines**, more **furnaces**, and the belts and inserters that connect them. From here the factory builds itself; you just keep feeding it goals.

Tip: The boiler must be fed even after it is running — a boiler that runs out of coal goes cold, and a cold boiler makes no steam. Until you have an electric miner belt-feeding it, keep a chest of coal and a burner inserter on it; the wiki is blunt that powered inserters “cannot operate” when no power is being produced, so the first boiler should always be hand- or burner-fed.

Hand-crafting vs machines. Pressing **E** and clicking a recipe makes it by hand while you stand there. Hand-crafting is free and instant-on-demand, but it costs **your** time and it is capped at one queue. The moment you need more than a handful of something, the rule is: **build a machine that makes it automatically**. The **assembling machine 1** is the first of these — it crafts a recipe on its own as long as inserters keep it fed, and you are free to go do something else.

Tip: Your first electricity milestone: one offshore pump, one boiler, one steam engine, two or three **electric mining drills** on the coal and iron patches, and a line of stone furnaces. The moment the drills and furnaces are electric, the burner loop’s manual feeding is over — you have graduated from castaway to apprentice.

Power & the Electric Grid

Before the belts get complicated, one detour pays for itself all session: how electricity actually behaves. Everything electrical draws from the same **grid** — power poles carry the wires, machines connect within reach of a pole, and the grid feeds whatever is attached. Understanding the grid answers most of the “why is nothing working” questions of your first hours.

Demand vs supply. The grid only produces what is being used — steam engines throttle up and down automatically as demand changes. If demand exceeds what the grid can produce, every machine slows to a crawl (a **brownout**), and once the grid collapses it can be hard to restart. The fix is almost always **more generation**, not fewer machines: another boiler, another engine, or more coal reaching them. Click any pole to open the electric network panel and see the real numbers.

Poles and reach. Small poles cover a short radius and are the cheapest way to extend power; medium poles reach further, and substations come much later. Power does not flow through a gap — a pole line that skips a link blackouts everything beyond it. The smart-placement trick: hold left mouse and drag while placing poles and they auto-spread at their maximum reach, so a power line lays itself along your walking path.

Steam vs solar. At Wave 0, steam is the answer: it is cheap, burns coal, and works day and night. Solar panels only produce in daylight and **accumulators** store power for night — both are researched later and become the better economics for huge bases, but a steam line is what carries the Castaway. The ratio to remember: one offshore pump line (200 boilers + 400 steam engines) is about 360 MW, which is more than an early base will ever draw.

The diagnosis, not the tool. The exact tools to ration power deliberately (power switches, circuit control) arrive in Wave 1. What Wave 0 needs is the diagnosis: a factory where every machine runs at half speed and the electric panel shows “no power” is a generation problem, and the cure is always more boilers or more coal.

Tip: Keep a buffer chest of coal next to the boiler line. Boilers eat coal continuously — a piece every 2.2 seconds each — and the moment the coal belt runs dry, the power fails, the miners stop, and the coal belt stays dry in a nasty deadlock. A chest of coal on the line is the insurance that breaks the cycle.

Automation Basics

Automation in Factorio means three machines working together: a **belt** carries things, an **inserter** moves things between them, and an **assembling machine** turns inputs into outputs. Understanding how these three behave is the entire game in miniature.

- **Transport belts** move items in one direction at a fixed speed — a yellow belt moves 15 items per second. Items sit on the belt waiting for a hand to pick them up. **Underground belts** dive under obstacles and **splitters** divide a belt into two, which is how one raw line becomes two demand lines.
- **Inserters** grab one item from behind them and place it in front of them. Point the green arrow at where it picks from, the yellow arrow at where it drops. A basic inserter swings a 90° arc every second or so; it can pull from a belt, a chest, or a machine, and push into any of those.
- **Assembling machines** hold one recipe each and craft it continuously while inputs arrive. The recipe is set from its panel (or copied with Shift + click). Assembling machine 1 is slow and handles only simple two-ingredient recipes — that is exactly what you need at Wave 0.

Turning a manual loop into a machine line. Take the red science loop as the worked example: an assembler needs **copper plates and iron gear wheels**. To feed it automatically, you build a line: a belt carrying iron plates past a gear-wheel assembler (an inserter pulls plates off the belt, the assembler makes gears, another inserter puts them back on a second belt), and alongside it a belt carrying copper plates. Feed both belts into a science assembler, and red science now produces itself

while you do anything else. This “belt feeds assembler, assembler feeds belt” pattern is the DNA of every factory you will ever build.

Belt patterns worth knowing. Belts are simple, but three patterns carry most of a factory’s traffic:

- **The supply line:** one belt carries a material past a row of consumers, and each consumer pulls what it needs with an inserter. This is how one iron plate belt feeds ten gear assemblers at once.
- **The crossing:** when two belts must cross, an **underground belt** dives under the other so neither line is interrupted. Never let one belt dump onto another by accident — a crossed belt is a corrupted belt.
- **The splitter:** a **splitter** divides one belt into two (or merges two into one). This is how a single smelter output becomes two separate demand lines, and it is the seed of the **main bus** that Wave 1 turns into an art form.

A worked example — the gear line. Iron plates on a belt, an assembler with the iron gear wheel recipe beside it, an inserter pulling plates in, an inserter dropping gears back onto a second belt running alongside. Two belts, one assembler, two inserters, one recipe copied from a neighbour. That is the smallest possible factory, and it is worth building by hand once so you can spot it instantly inside any bigger build — because the bigger builds are just the same cell repeated.

Tip: Read the arrows. Every belt, inserter, pipe and underground has direction arrows that show in alt-mode. A whole factory “not working” is, nine times out of ten, one inserter or belt pointing the wrong way — the **Bottleneck Lite** mod from the Modlist chapter turns a red dot onto the exact machine that is starved, which makes these fixes take seconds.

Warning: Do not hand-feed assemblers from your inventory for long. An assembler that is fed by **you** stops every time you walk away, and walking away is the whole point of a factory. If a machine line needs an item, build a belt or a chest that provides it — the moment your hands leave the loop, the loop is real.

Tip: Use the copy-paste tools relentlessly: **Shift + right-click** an assembler, then **Shift + left-click** a row of empty ones to give them all the same recipe. A row of ten machines configured in two clicks beats ten machines configured one-by-one every time.

Science Packs & Research

Everything you are allowed to build is locked behind **research**, and research is paid for with **science packs** fed into **labs**. The entire early game is one loop: automate a science pack, feed it to labs, and research unlocks the machines that make the next, harder science pack.

- **Automation science (red):** 1 copper plate + 1 iron gear wheel, crafted in an assembler (5 seconds per pack). This is your first fully automated line — the worked example in the last section. Red science unlocks basic automation, the assembler-2 upgrade, and the next science tier.
- **Logistic science (green):** 1 transport belt + 1 inserter (6 seconds per pack). Green science needs a second assembler line that makes belts and inserters — both things you already use — and it unlocks belts upgrades, splitters, and the logistics items.
- **Military science:** 2 packs per crafting cycle from gun turrets, piercing rounds and stone walls. Optional until the biters force the issue; it unlocks the defense tree.
- **Chemical science (blue):** the first science that drags in oil — plastic bars and sulfur come out of the petroleum chain. Blue science is the moment the factory stops being about plates and starts being about chemistry (next section).

Labs are the consumers: build a row of them, feed every science pack you make into the same line, and select a research from the technology screen. The **research queue** lets you line up several technologies — always keep the queue full so the labs never idle.

Building the red-and-green line. The classic Wave 0 science build is two cells side by side. For red science: an iron-plate belt feeds a row of gear assemblers (gears back on a belt), and that gear belt plus a copper-plate belt feed a row of science assemblers. For green science, the same shape: a belt of transport belts and a belt of inserters feeding another science row. Feed all the science belts into the same lab line and the labs swallow red and green together. It is the Automation Basics pattern, repeated twice, and it produces a **continuous stream** of both packs instead of a trickle — the difference between “research takes forever” and “research just happens”.

Tip: The research screen is the game’s roadmap. Click any technology and its tooltip lists the exact science packs it needs and what it unlocks. Rule of thumb for the first hour: **automation** first (unlocks assembler lines and red science), then **logistics** and **automation 2** for belts and inserters — and keep red and green science feeding labs continuously from the moment you can.

Warning: Do not research everything at once. Science packs take real resources, and spreading thin means nothing gets done. Pick a goal (better belts, red science, the next science tier) and research toward it — the queue is for chaining related goals, not for spending every spare pack.

Tip: The **Factory Planner** mod from the Modlist chapter is built for this moment: instead of guessing how many assemblers a science line needs, plan it — set a target (say, one science pack per second) and it tells you exactly how many gear assemblers, plate furnaces and miners the chain needs. **Rate Calculator** measures the answer for lines you have already built. Together they end ratio-guessing forever.

Oil Processing

Blue science demands **plastic**, and plastic demands **crude oil** — which means you must find oil, pump it, and process it. Oil is the first resource that does not come out of the ground as a usable product.

Finding and pumping. Crude oil sits in pools of black liquid on the map, usually a little way from your base. Place a **pumpjack** over a well (pumpjacks need electricity, unlike burner drills) and it extracts crude. Pipes — laid like belts but for fluids — carry it to the refinery. Note that pumpjacks run at a fixed rate that declines as the well depletes; the **YARM** mod in the Modlist chapter can track how fast each well is draining.

Refining. A **oil refinery** runs basic oil processing: it turns crude oil into three fractions at once — **petroleum gas**, **light oil**, and **heavy oil**. All three are useful, but petroleum gas is the one you need first, because plastic bars come from it, and plastic is blue science’s gate. If your heavy and light oil tanks fill up while you still want petroleum, the refinery stops — that is the classic oil problem.

Cracking. The answer is **cracking**: chemical plants convert the heavier fractions into lighter ones — heavy oil into light oil, light oil into petroleum gas. Cracking is researchable and turns your crude oil almost entirely into petroleum when you need it. The flow is: **pumpjack** → **refinery** → **storage tanks** → **crackers (heavy → light → petroleum)** → **plastic plant** → **sulfur** → **batteries**. Batteries and rocket fuel — both needed much later for the rocket — are also oil products.

The plastic chain, step by step. Plastic bars are the whole point of the early oil phase, and the chain is short enough to hold in your head:

1. Pump crude with **pumpjacks** and pipe it to a **oil refinery** set to basic oil processing.

2. Pipe the petroleum gas output into a **chemical plant** making **plastic bars** (petroleum gas + coal).
3. Pipe more petroleum into a chemical plant making **sulfur**, which turns into **sulfuric acid** for batteries and blue science's needs.
4. When your heavy and light oil tanks creep toward full, add the **cracking** recipes so the excess flows downhill to petroleum.

That one refinery block unlocks chemical science, and chemical science is the gate to everything that follows — batteries for mining productivity and utility science, plastic for advanced circuits and low density structures.

Tip: Build storage tanks as buffers before every consumer, and crack only when a tank is nearly full. A simple rule that prevents 90% of oil deadlocks: if heavy oil is above half a tank, crack heavy → light; if light is above half, crack light → petroleum. Your refinery line then never jams on a full tank.

Warning: Oil refineries and chemical plants are **fluid** machines: they connect by pipes, not belts. Check that every pipe's arrows point the right way and that nothing is fed by an inserter — a pipe with no connection looks almost identical to a working one, and a refinery with full outputs quietly stops. **Alt-mode** shows fluid levels in the pipe tooltips.

Early Rail

The castaway's instinct is to build everything next to everything else. By the time your iron demand outgrows the starter patch, a train line becomes the better answer — and the moment it earns its place is the moment hauling by belt would mean dragging ore halfway across the map.

When a train justifies itself. If your mining outpost is farther than your yellow belts and underground belts comfortably reach, or you are hand-walking coal and ore back to the base, it is rail time. A single **locomotive**, one or two **cargo wagons**, and a loop of track is the whole system — the train hauls ore from a far patch to your base, and fuel (coal works fine, solid fuel better) keeps it running.

Track layout. Rails connect in two basic ways: a **two-way line** with a turnaround loop at each end (simplest, good for one train), or a **one-way pair** of parallel rails for throughput. Place **train stops** at the loader and the unloader, give them distinct names ("Iron Mine", "Iron Drop"), and set the locomotive's **schedule**: go to Iron Mine until full, go to Iron Drop until empty. Train **limits** on each stop (a number in the stop's panel) keep one train per station and stop pile-ups.

Signals. With a single train you barely need signals. With two or more on shared track, you need them: **rail signals** divide the track into blocks and a train only enters an empty block; **chain signals** read ahead and are used before crossings. The rule every beginner needs: signal the track so a train always has a green block ahead of it, and put a chain signal anywhere a train could wait on a crossing.

Designing your first line. Here is a line that earns its keep in an afternoon: a coal or iron patch a couple of screens away, plus a hungry smelting block back at base.

1. Lay a single rail track from a **train stop** at the mine to a **train stop** at the base, with a turnaround loop (a big "O" of track) at each end.
2. Name the stops "Coal Mine" and "Coal Drop". Place the locomotive, one or two cargo wagons, and one more locomotive facing the other way if you want symmetric acceleration.
3. Open the locomotive and set the schedule: **go to Coal Mine, wait until cargo full; go to Coal Drop, wait until cargo empty.**
4. Feed the mine stop from electric miners and unload the base stop into chests with fast inserters, then belt the ore onward.
5. Add a second train when the first can't keep up, and signals where the two share the main line.

A working single-train loop moves more ore per hour than you will ever hand-carry, and it is the smallest thing in Factorio that feels like a real logistics system.

Tip: Name your stops clearly and keep schedules simple. The built-in schedule editor is forgiving — a train with two stops and the ‘until full / until empty’ conditions runs itself forever. Start with one train and a loop; a working loop is worth more than a sprawling network that you never finish.

Warning: Trains do not stop for you. Standing on a rail is a death sentence, and at Wave 0 there is no respawn-at-base shortcut for a crushed engineer. Build a footbridge over or a gap under your tracks, and treat live rails as walls until you have crossed safely.

Biters & Defense

Nauvis is not empty. **Biters** are the native life, and they are the only thing on the planet that actively wants your factory destroyed. The good news: they are predictable, and defense is a solved problem. **Pollution is the trigger.** Every machine you build emits **pollution**, a cloud that spreads across the map (each chunk absorbs and disperses it; trees soak up some too). When that cloud reaches a biter nest, the nest’s **spawners** absorb the pollution and use it to muster biters into attack waves, launched every 1 to 10 minutes against the source. Build quiet, and the natives leave you alone; build loud, and they come.

Evolution is the difficulty dial. The **evolution factor** — the world’s overall threat level — creeps up from three sources: the passage of time (slow), the total pollution you produce (faster), and **destroying nests** (the biggest jump by far). Higher evolution means bigger, tougher biters. The practical takeaway: do not go nest-clearing until you have real weapons, because every nest you kill makes the whole world harder.

The basic defensive line. A ring of **gun turrets** fed with **magazines** (a chest or belt of ammo behind them) backed by **walls** will hold off the early waves. Turrets are cheap and their ammo is cheap; wall segments are cheap. Repair packs fix the damage between waves. When you can, upgrade turret ammo (piercing rounds come from military science) and add **flamethrower turrets** for the wall-crawlers. Place your line **outside** the base footprint and leave a gap so you can reach the walls to repair them.

Knowing your enemy. Biters come in escalating sizes — small, medium, big, behemoth — and **spitters** lob corrosive fluid over your walls, which is exactly why the walls exist: they give the spitters something expensive to chew while the turrets shoot back. **Worms** guard nests and will hit you if you stroll up to clear them. The evolution dial decides which sizes appear: early on the attacks are small biters that a gun turret eats for breakfast; late in the game they are big and behemoth hordes that demand real firepower. Reading the pollution map (red overlay on **M**) shows you how far your pollution cloud extends — and whether it is reaching biter nests; reading the evolution bar — hover the **Enemies** entry on the map — tells you how hard the next attack will be.

Tip: Watch the pollution cloud on the map (press **M** and check the red overlay). If the cloud is still short of the nearest nest, you are safe; the moment it touches a spawner, expect an attack within the hour. Use the radar to watch both.

Warning: The pistol is a tool of desperation, not a plan. If biters overrun your wall, the correct response is not to stand and shoot — it is to retreat, repair the gap, and build more turrets. A wall with turrets behind it beats any amount of personal firepower at Wave 0.

Launching the Rocket

Everything in The Castaway leads to one building: the **rocket silo**. This is the wave's goal — not a new game plus, not an expansion ending, just the vanilla victory condition you can reach entirely with the systems in this chapter.

The research gate. The Rocket silo technology takes **every science pack except military** — automation, logistic, chemical, production, and utility. By the time you research it you have already built the whole game's economy: the science chain that made blue science is the same chain that makes production and utility science, just with more steps and more machines.

The parts. A rocket needs **100 rocket parts**, and since 2.0.7 each part costs 10 **low density structures**, 10 **processing units**, and 10 **rocket fuel** (crafted in 3 seconds). That is 1,000 of each ingredient per rocket — a serious but entirely mechanical logistics problem: LDS from copper/steel/plastic, processing units from the circuit chain, rocket fuel from oil. Feed all three to the silo and it assembles the parts automatically.

The three belts. If the ingredient list feels like a wall, reduce it to the three production chains it actually is:

- **Low density structures** (copper + steel + plastic) — your smelting lines and the plastic plant from the oil section, scaled up. This is the “heavy metal” belt.
- **Processing units** (electronic circuits + advanced circuits + batteries) — the circuit chain you have been building since green science. This is the “brains” belt.
- **Rocket fuel** (solid fuel from light oil and petroleum) — the oil refinery from the same section. This is the “thrust” belt.

Each belt is already something you built for an earlier goal; the rocket just asks for a lot of it. When all three converge on the silo, it assembles a part every few seconds, and a hundred parts later the launch button lights up.

Note: This whole chapter assumes the base game, because Waves 0-1 run with the Space Age expansion disabled. If you do fire up a Space Age-enabled save, the rocket is a very different beast — the silo unlocks far earlier and a rocket part costs 1 of each ingredient instead of 10. The guide will say exactly when and how in Wave 2, where Space Age comes back on. For The Castaway, the base-game recipe above is the one to learn.

The payload. Before the first launch you need a payload: a **satellite** (the classic choice). Launch with a satellite and the rocket returns **1,000 space science packs** to your cargo landing pad about 29 seconds after launch — the game's reward loop for launching. The first launch triggers the victory screen.

Tip: The rocket is the capstone, not the stopping point. After it launches you are welcome to keep growing — space science opens the infinite research and the post-rocket game. But for The Castaway, the launch is the win: every system you built on the way is the factory you just proved. Wave 1: The Architect takes that proof and makes it scale.

Warning: Don't let the rocket's ingredient list scare you. It is the same three belts you already built — copper-and-steel for LDS, circuits for processing units, oil products for rocket fuel — just scaled up. If a line stalls, the **Production statistics** (P) screen shows you which one, and **Rate Calculator** tells you how many machines are missing. The rocket is a logistics check, not a wall.

Common Beginner Mistakes

- **Spaghetti paralysis.** Waiting for a “perfect” layout before building anything. The factory you can build now, badly, produces more than the perfect one you never place. Build it, and rebuild it later — deconstruction is free, and blueprints (B) make moving a production block a minute of work.
- **Ratio blindness.** Building one of everything and wondering why nothing keeps up. Every recipe has a ratio: how many miners feed a furnace, how many furnaces feed an assembler, how many assemblers feed a science line. You can count them on paper — or let **Factory Planner** and **Rate Calculator** do it for you. “One of each” is not a plan.
- **Hand-feeding.** Running around stuffing chests and machines from your inventory instead of building belts. Every time you hand-feed, the machine stops when you leave. The fix is always the same: a belt, an inserter, or a chest — never your hands.
- **Ignoring pollution.** Not watching the map, and being surprised by the first attack. If the cloud touches a nest, biters are coming. Watch the map, keep turrets and ammo stocked, and treat the first attack as a scheduled appointment, not an ambush.
- **Skipping alt-mode.** Playing half-blind. Press Alt once at the start and the whole factory becomes readable — the single highest-value keystroke in the game.
- **Building on the ore.** Placing your smelting line on top of the very patch you need to mine. Leave the patch clear for miners; build the factory beside it, not on it.
- **One-way belts everywhere.** Running a single belt line out to a distant outpost and back instead of two parallel belts or undergrounds. Dedicated lanes — one out, one back — keep throughput clean and debugging easy.
- **Ignoring the grid.** Treating a brownout as a mystery. If every machine runs at half speed and the electric panel says “no power”, that is generation shortfall — add boilers or coal, not more machines. Diagnosing power first clears half of all “nothing works” moments.
- **Starving the boilers.** Forgetting the power line eats coal every second. A boiler without coal makes no steam, the grid dies, and the miners that would have refuelled the boiler are dead too — the classic deadlock. A buffer chest of coal on the boiler line is cheap insurance.
- **Expanding into the biters.** Clearing nests “because they’re there”. Destroying spawners is the single biggest evolution jump in the game; only clear the nests that threaten your pollution cloud, and only when your turrets can hold the new frontier.

None of these mistakes are fatal. Factorio expects you to make them, fix them, and come back stronger — that is the loop. By the time the rocket leaves the pad, you will have made all of them and learned from each, and that is exactly what The Castaway is for. The factory grows, and so do you.

The Castaway's First Session

If you want to close the guide with your hands on the keyboard, here is a single sitting that touches every system in this chapter and leaves a factory that outlasts the session:

- **Minute 0–20:** work the **first ten minutes** script from **Your First Minutes** — burner loop, then electricity. Build two or three electric drills on coal and iron and a short line of stone furnaces.
- **Minute 20–40:** build the **Automation Basics** cells — a gear assembler fed by belts, then a red science assembler fed by gears and copper. Two labs on the same belts. Research **automation** and **logistics**.
- **Minute 40–60:** expand to **green science** — belt and inserter assemblers feeding the same lab line. Queue the research. Watch the pollution cloud on **M** to see how close the nearest nest is.
- **Minute 60+:** call the session done when red and green science are streaming into labs unattended. Everything after that — oil, rail, the rocket — is the same loop repeated at higher stakes.

End the session with the factory still running: science producing, drills mining, labs researching. That is the castaway's definition of progress, and every future session in this guide is just that loop made bigger.

The Castaway: Modlist

Wave 0 adds no content. Every mod in this chapter only makes the game easier to see, easier to operate, or easier to plan — nothing here changes a recipe, a balance number, or a research tree. Think of it as servicing the console before you touch the machinery. All seven mods are verified on the official Factorio Mod Portal against Factorio 2.0.77: each card records the mod's last-update date, its 2.0-line build, and its dependencies, and every one of them passes the power-spike test untouched. The seven mods are split across the next two sections by what they change. **Mechanics** covers the five system mods — Even Distribution, Squeak Through 2, Rate Calculator, Factory Planner and YARM — quality-of-life and planning tools that change how you interact with the factory, not how the factory behaves. **Graphics** covers the two visual and interface mods — Todo List and Bottleneck Lite — how the game looks and how it tells you what is happening. Not one of them touches output caps, throughput, efficiency or costs, so the whole wave passes the power-spike test without a single borderline verdict.

The Castaway: Mechanics

The five system mods in this section change how you drive the factory, not what the factory produces. Three of them are pure convenience (walking, stacking, distributing), two are pure information (measuring and planning), and none of them moves a single balance number. If a mod ever started changing recipes or caps, it would fail this wave's one rule — and every card below carries the evidence that it does not.

Even Distribution

Version/Updated: Updated 24 Jun 2026 (page); 2.0-line build v2.0.2 (22 Oct 2024). Newer 2.1-line releases (2.1.0, Jun 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Even Distribution' matches the API title.

Dependencies: None — standalone.

Impact: Hold Ctrl and drag a stack across several machines, chests or wagons and the items split evenly between them; Shift+C flushes spare inventory items into the nearest buildings. Vanilla 2.0 can dump a stack into one entity at a time — this distributes it across a whole row. Pure convenience: nothing about recipes or production numbers changes.

Notes: By 321freddy; 517K+ downloads; the long-standing community standard for filling many buildings in one gesture. The author is actively maintaining the mod (the 2.1-line build shipped Jun 2026); a 2.0.77 install automatically picks the newest 2.0-line build, v2.0.2.

Squeak Through 2

Version/Updated: Updated 23 Jun 2026 (page); 2.0-line build v0.1.5 (23 Apr 2026).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Squeak Through 2' matches the API title.

Dependencies: None — standalone.

Impact: Lets the engineer walk — and later drive — between buildings, pipes and machines that would normally block movement, so a dense factory stops being an obstacle course. Movement and collision QoL only; no machine, recipe or balance value is touched.

Notes: By _CodeGreen; 270K+ downloads. A from-scratch, highly configurable 2.0 rewrite of the classic Squeak Through — the original mod (by Nexela) was not carried forward to 2.0, and this is its 2.0 successor. The 2.0.77 install uses v0.1.5.

Rate Calculator

Version/Updated: Updated 14 Jul 2026 (page); 2.0-line build v3.3.8 (18 Jan 2026).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Rate Calculator' matches the API title.

Dependencies: None — standalone.

Impact: Drag a selection box across any machines, miners or belts and it reports exactly what the build produces and consumes per second, and which machines are running below full throughput. Purely informational — it reads the factory and never changes it, so it cannot raise or lower any cap.

Notes: By raiguard; 426K+ downloads. Complements Factory Planner: Rate Calculator measures what already exists, Factory Planner designs what comes next. Running the two together is the standard planning stack. The 2.0.77 install uses v3.3.8.

Factory Planner

Version/Updated: Updated 1 Aug 2026 (page); 2.0-line build v2.0.50 (19 Apr 2026).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Factory Planner' matches the API title.

Dependencies: None — standalone.

Impact: Adds a spreadsheet-like planner that works out the whole chain — inputs, machines, ratios, speed and module choices — for any target throughput before you place a single building. It plans on paper only: nothing is placed in the world and no game value is touched.

Notes: By Therenas; 329K+ downloads. This is the tool that turns 'ratio blindness' into a solved problem from day one. Pairs with Rate Calculator (measure what exists, plan what comes next). The 2.0.77 install uses v2.0.50.

YARM - Resource Monitor

Version/Updated: Updated 1 Jan 2025 (page); 2.0-line build v1.0.5 (1 Jan 2025) — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'YARM - Resource Monitor' matches the API title.

Dependencies: None — standalone.

Impact: Tracks every ore patch you mark, showing percent mined and estimated time to depletion, and warns you when a patch is about to run dry. Vanilla 2.0 shows a patch's remaining amount when you hover it; YARM extends that into persistent tracking, a history graph and

alerts, so a dying iron patch never ambushes you. A monitoring aid only — it cannot change a patch or a miner.

Notes: By Narc; 168K+ downloads. Its single 2.0-line build (v1.0.5, Jan 2025) is also its newest release on any line, so there is no version divergence to watch for on 2.0.77.

The Castaway: Graphics

The two mods in this section change how the game **looks** and how it talks to you — the interface panel and the status dots on every machine. None of them changes what the factory produces; they change how clearly you can see it. Wave 0 keeps every recipe and balance number exactly as vanilla ships them.

Todo List

Version/Updated: Updated 28 Jun 2026 (page); 2.0-line build v19.15.2 (5 Jun 2026).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Todo List' matches the API title.

Dependencies: None — standalone.

Impact: Adds a checklist panel for planning and tracking tasks — research goals, expansion jobs, things to fix. In multiplayer the list is shared, so a co-op crew can synchronise who builds what and nothing gets forgotten. A pure interface addition; no gameplay value is changed.

Notes: By JasonMiles; 275K+ downloads. The panel is bound to any key you like in the controls menu; pick one you will actually press. The 2.0.77 install uses the 2.0-line build v19.15.2.

Bottleneck Lite

Version/Updated: Updated 23 Jun 2026 (page); 2.0-line build v1.3.4 (6 Jun 2025).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Bottleneck Lite' matches the API title.

Dependencies: None — standalone.

Impact: Shows a small coloured dot on every machine — green while running, yellow when full or waiting on output, red when starved of input — so a dying belt is visible from across the factory instead of hiding inside a tooltip. It runs with zero runtime overhead and updates instantly. Display-only: it changes nothing about how machines behave, so no cap or cost is touched.

Notes: By raiguard; 222K+ downloads. The original Bottleneck mod does the same job but stopped updating in Dec 2024; Lite is the actively maintained 2.0 choice (its 2.1-line build shipped Jun 2026). The 2.0.77 install uses v1.3.4.

Wave 1: The Architect

The Architect: Strategy

The Castaway proved that a factory can be built. The Architect's job is to prove that it can **scale** — that the bus survives its own width, that the rail network carries a dozen trains without a single deadlock, that circuits make the factory decide things instead of you, that robots and modules and quality pay for themselves. This chapter teaches the eight systems that scaling is built from, in the order a growing factory meets them: the bus, the rail network, circuits, robots, modules, quality, the mathematics, and finally the city-block capstone. Every number below was checked against the official Factorio wiki for 2.0. Everything here describes the base game, which is how Waves 0–1 run (Space Age stays off until Wave 2).

The Bus: Order Out of Spaghetti

There are two ways a factory grows. **Spaghetti** is what a factory becomes when every building is placed where the materials happen to be — fast to build, hellish to expand, and impossible to reason about once a belt crosses a belt crosses an inserter. The **main bus** is the cure: a small number of broad, parallel belts that carry the core materials past every production block, with each block pulling off what it needs. The moment the base outgrows its first area, the bus is how the Architect buys future-proof space.

What goes on the bus. The rule is simple: carry the items that **many** blocks consume, not the items one block makes. The classic spine is, roughly in order: iron plates, copper plates, green circuits, steel, and then plastic, red circuits, and stone/brick as the mid-game adds them. Each of those deserves its own dedicated lane(s), because each is consumed in every direction. What does **not** go on the bus is anything with a single consumer — a purple science ingredient used in one block belongs in that block's own feed, not on the highway.

Lane count is the plan. Reserve lanes when you lay the bus, not when it fills. A common scale is 4 lanes of iron, 4 of copper, 2 of green circuits, 2 of steel, and singles for the rest — and the bus is laid **two tiles apart per pair** so splitters and undergrounds can cross without breaking. Remember the numbers from the last chapter: a yellow belt carries 15 items per second, red 30, blue 45 (two lanes total on each belt, both sides of the belt count). If your iron consumption is heading past 60/s, that is four yellow belts or two blue — and you should already have the space reserved.

Where to expand. Always expand the bus **lengthwise, then sideways**. Production blocks reach out from the bus and get longer; the bus itself keeps running straight. When it must fill with a new lane, the clean move is to add the lane on the outside rather than tear up the middle. Balancers matter, but they are a later refinement — for the Architect, the discipline of **dedicated lanes, pulled off by splitters, never dumping one belt onto another** prevents nine-tenths of bus failures.

Tip: The bus is a **policy**, not a religion. A 'lane' is one belt; two parallel belts with items on only one side of each can carry as much as one full belt, so keep each lane saturated on both sides and pull off only what a block needs. When a block's demand approaches a full lane, split the lane **early** and let both halves feed — a single splitter at the start beats starving consumers at the end. The **Prismatic Belts** mod colors each lane's direction, which turns a four-lane bus from a guessing game into a diagram. **Belt Visualizer** highlights a whole connected line so you can find the one empty stretch starving a block.

Warning: Never pull items off a bus and insert them back onto the same lane — that is how a bus corrupts. Feed a block from the bus and return its waste down a dedicated return line. And build the smelting before the lanes it feeds: the bus is only as good as the smelters behind it.

Trains: Signals, Blocks, and a Network That Scales

Wave 0 gave you a single train loop. The Architect runs **networks** — multiple trains, shared track, crossings, junctions — and that is where signals earn their keep. The two laws of rail are: a train never enters a block it is not allowed to enter, and every crossing must be guarded by a chain signal.

Blocks. Every rail signal divides the track into **blocks** — the segments of rail between signals. A train only enters a block whose entry signal is green; once inside, all the block's entry signals turn red until it leaves. That one rule is the entire safety model, and it scales: more signals means smaller blocks means more trains can share the same stretch of track safely.

Chain signals read ahead. A chain signal copies the state of the **next** signal along the train's path. Place it on the entrance to a crossing or junction, and a train will not enter the crossing unless it can also leave it — which is exactly what stops a train from parking across an intersection and deadlocking everyone else. The special case worth knowing: if a chain signal can reach its destination (a stop) without crossing another signal, it lets the train through regardless. In practice the rule is short: **rail signal on every exit, chain signal on every entrance, chain signals before any crossing.**

Station naming and train limits. A train schedule is written against stop **names**, so names are a language: name a stop by what it does ("Iron Ore In", "Iron Ore Out", "Drop"), and the same name on two outposts means one train serves both. The **train limit** — a number set in the stop's panel — caps how many trains may head for a stop at once, which is the single best anti-deadlock habit in the game: one train per station until you have a reason for more.

Scaling the network. When two trains share a line, signal it as two one-way tracks (each with its own signals) instead of one two-way line. Put signals far enough apart to fit the longest train. And when the schedule bookkeeping starts to feel like the actual work, that is the moment the **Project Cybersyn** mod from the Modlist chapter steps in: it turns stops into providers and requesters driven by circuit signals, and a small fleet of trains runs itself.

Tip: The deadlock test before you trust a junction: with your longest train parked inside the crossing, there must be enough room after each chain signal for the whole train to clear the junction. If a train can't physically fit in the next block, shorten the block or move the signal. And put a signal after every train stop so the station itself is its own block — otherwise the train behind the one unloading blocks the whole main line.

Warning: A two-way single track with more than one train is a deadlock factory. The honest scaling move is one-way pairs: two parallel rails, each signalled in one direction. It costs a little more track and buys unlimited growth.

Circuits: The Factory's Nervous System

The circuit network is the difference between a factory you **run** and a factory that **runs**. Wires read values, combinators decide, and machines listen. It is unlocked by the circuit-network research, and its units are **signals** — numbers attached to item names or to the virtual signals (letters like A, B, C and specials like the checkmark).

Wiring. Red and green wire each form their **own** network: two red networks never merge unless they physically touch, and a red and a green network on the same pole stay separate. That is the feature that lets you run two independent ideas down the same row of poles. Signals **sum** — wire

three chests of iron ore together and the network reports the total; a receiver connected to both a red and a green wire sees the sum of both.

The three combinators. The **constant combinator** broadcasts up to 20 fixed values — it is the factory’s memory, your way of saying ‘this station wants 4,000 iron ore’. The **arithmetic combinator** takes an input and outputs a calculated result (multiply, divide, add, subtract, even modulo). The **decider combinator** compares a signal to a value or to another signal and passes a chosen signal through when the condition is true. Note that an arithmetic or decider combinator’s input network and output network are **separate** — the output is broadcast on the other side, which is exactly how you build feedback loops.

Three real applications. Three builds cover most of what a scaling factory needs:

- **Enable-by-threshold.** Read a chest’s contents with a wire, feed the signal into a decider set to ‘iron ore > 4,000, output checkmark = 1’, and connect the output to a train stop set to ‘enabled when checkmark = 1’. The train only comes when the buffer actually needs it.
- **Request-from-circuit.** Connect a requester chest (or a warehouse) to the circuit network and set it to request whatever a constant combinator says — one combinator flips the whole line’s recipe.
- **Count and divide.** An arithmetic combinator reading a belt can count items; dividing a stock level by the stack size turns a count into ‘how many trains fit’.

Tip: Combinators are slow, dumb, and wonderful: they update once per tick and they never guess. Start with **one** real circuit — the enable-a-train-stop threshold above — and learn to read it with the **Circuit Visualizer** mod, which draws the signal values right on the wires, and **Nixie Tubes**, which show a signal as digits on the wall. When you can read what a single network is doing, building the second and the third becomes pattern-matching. The **Circuit HUD V2** mod pins the same numbers to your screen so a city block’s buffers are visible while you stand anywhere.

Warning: A circuit network only knows what you **wire** into it — it never sees the contents of a chest that is not connected, and a wire that looks connected but is not is the most common circuit bug in the game. And remember that receivers sum red and green: if a build behaves twice as strong as you intended, a red and a green wire are probably both reaching it.

Logistics Robots: When Bots Beat Belts

Belts are the backbone, but they are **dedicated**: a belt line carries one thing in one direction and its capacity is spent the moment you place it. The **logistic network** is the flexible layer — a shared pool of items that any requester can draw from, delivered by flying robots. The trade-off is the whole Architect’s art: belts for bulk, bots for **mix**.

The parts. A **roboport** creates a logistic zone (a 50x50 area, with a larger 110x110 construction area) and is where robots rest and charge; roboports whose orange zones touch join into one network.

Passive provider chests offer contents to the network but do not push them out; **active provider chests** push their contents out immediately; **storage chests** hold the network’s loose items (and are where deconstruction drops things); **requester chests** ask for specific items; **buffer chests** do both — they request and provide. Logistic robots deliver: by default each carries a single item and flies at about 3 tiles per second, and the **worker robot speed** and **worker robot cargo size** researches raise both, the cargo limit to four items per robot.

How robots choose. Robots pick up in a fixed priority — active providers first, then storage and buffer chests, then passive providers — so the network always drains an active provider before touching your careful passive stacks. A requester’s demand is filled by robots that pick up the item and fly it over; when demand exceeds supply, the network thins out, and the answer to ‘why is my build slow’ is usually not enough bots, not enough providers, or too long a flight.

When to switch from belts. The rule that never fails: keep **bulk, high-volume, one-directional** flows on belts and trains — plates, ore, circuits by the belt-load — and give **high-mix, low-volume** flows to the bots. Green science packs, modules, inserters, repair packs, construction materials: dozens of different items, each needed in small numbers, in changing places. That is what the logistic network is for. A single requester chest beside a science assembler with a provider chest of plates upstream replaces a dedicated belt line and every splitter on it.

Tip: The **Warehousing Mod** from the Modlist chapter is the robot network's megabase upgrade: a 6x6 warehouse can be a passive provider, storage, requester or buffer chest, so a whole city block's buffer becomes one giant chest instead of a field of steel ones. Pair it with a requester-warehouse feeding an assembly block and the block becomes self-provisioning — request what it needs, provide what it makes, and the network settles the accounts.

Warning: Robots are a **network**, not a conveyor: every robot crossing a network is one that is not doing something else, and a long-distance bot haul is slow, power-hungry logistics. A common scaling failure is one giant network for the whole base — bots fly forever and charging pads jam. The fix is not more bots, it is more networks: keep each bot network small and local (one block, one production cell), use belts and trains between cells, and let robots work only the short, high-mix hops inside them.

Modules & Beacons: Spending Spare Throughput

Modules are the Architect's lever for turning **materials** into **speed**, and they are expensive on purpose — the wiki's own framing is that they are costly, research-gated and slowly produced. Every module is a trade, and the whole module system is about choosing which trade.

The three families. **Speed modules** (1/2/3: +20%/+30%/+50% crafting speed) make a machine act like several machines, but each also raises power draw and pollution by more than the speed gain. **Efficiency modules** (1/2/3: -30%/-40%/-50% energy) cut the power and pollution bill, ideal for drills, beacons and any always-on line. **Productivity modules** (1/2/3: +4%/+6%/+10%) make the machine create bonus items out of nothing on every cycle — the only 'free' material in the game — but slow the machine and raise its power draw. Module effects are **additive**, not multiplicative, and energy use can never be reduced below -80%.

Beacons. A **beacon** transmits the effects of its own modules to every compatible machine in a 9x9 area around it. It can push a machine past the limits of its own module slots, but its effect has built-in diminishing returns — the more beacons hammer one machine, the less each one adds. The standard mid-game shape is a row of assemblers (each packed with productivity modules) flanked by beacons full of speed modules: the productivity creates free items, the speed beacons cancel the productivity slowdown. That pairing is the single biggest throughput lever in the base game, and it is also why 'prod in the machine, speed in the beacon' is the first module build worth learning.

Tip: Productivity modules pay for themselves in **raw material** but cost you in machines: a prod-3-filled assembler makes 10% free items but runs half speed. The whole point of speed beacons is to buy that speed back — but beacons themselves consume power constantly, even when the machines they boost are idle, so put a power switch on a beacon array that serves a line you only run occasionally. And when a build needs a module recipe changed across fifty machines, the **Module Inserter** mod lets inserters and robots handle the slots instead of your hand.

Warning: Speed modules on drills, miners and pumpjacks burn power and pollution without adding new material — the ore was coming out anyway, just slower. Efficiency modules are almost always the right call on mining and on the boiler/steam line: a –50% power machine in a field of hundreds is a whole power plant saved. Reserve speed and productivity for the **assembling** economy, where the multiplier actually multiplies.

Quality: Better Things, Not Just More

Quality is 2.0's second axis of scaling. Every item can come out at five tiers — normal, uncommon, rare, epic, legendary — and higher tiers are strictly better: a legendary assembling machine crafts 30% faster per tier, a legendary module boosts its effect by 30% per tier, a legendary chest holds more. The trick is that quality is **gambled**, not chosen: a machine fitted with **quality modules** has a chance (quality module 1/2/3: +1%/+2%/+2.5%) to roll a higher tier on every craft, and the **recycler** turns items back into a quarter of their ingredients — so you can loop a build, recycling normal output and keeping whatever rolls higher.

When quality pays off. Quality is not always worth it; it is worth it when the **same item is built in huge volume and used forever**. The classic targets are machines and modules themselves: one legendary assembler, beacon or speed module is a permanent, compounding upgrade, so a dedicated quality loop that slowly accumulates legendary machines pays for its up-front cost many times over. The same logic applies to buildings you place in bulk — but **not** to consumables like science packs, where the odds are better spent on volume, and **not** to every line in the base.

The upcycling loop. The practical pattern is **craft-normal, recycle-waste**. A bank of machines with quality modules crafts, say, iron plates; normal output feeds the factory, uncommon and better get sorted into a loop that crafts them into **higher** tier intermediates — quality compounds on quality. Because the recycler returns only a quarter of its input (and rolls quality again), the loop is deliberately expensive: it spends material to pull higher tiers out of the odds. That material cost **is** the gate — quality upcycling is the game's own version of 'pay resources to buy luck', which is exactly the kind of proportionate cost this wave's rules demand.

Tip: Sort quality by belt, not by hand: inserters have had filters since 2.0.7, so a line of filter inserters can split uncommon from normal onto separate lanes, and a filter-splitter arrangement pushes each tier toward its own destiny. Start with the **small** loop — quality modules in a machine making its own inputs, recycle the normal output — and only scale it once you can read the numbers in the **Circuit HUD** and manage the buffers.

Warning: Do not quality-module your whole base. Quality modules slow machines down and add nothing to a line that is already feeding a hungry consumer — they are an investment, not an amplifier. If a line's normal output is fully consumed, leave it normal; quality pays only where you are deliberately **banking** better items for future builds. And do not pair quality modules with speed modules in the same machine or its beacons — speed just churns out more normal items while the quality chance stays fixed per craft, diluting the odds; the wiki's own guidance is to keep quality modules away from speed.

Rate Mathematics: Counting the Factory

A scaling factory is a set of equations, and the Architect's superpower is doing them in the head or on paper. The core formula is small:

The core formula: items per second = (machine crafting speed) × (1 + speed bonus) ÷ (recipe time in seconds) × (1 + productivity bonus)

A machine's **crafting speed** is its multiplier on the recipe's base time. An assembling machine 2 runs at 0.75; an assembling machine 3 at 1.25. A recipe that takes 1 second in an assembler 2 therefore makes 0.75 items per second, and a recipe taking 0.5 seconds makes 1.5 per second. Speed modules push the 1 + speed bonus term; productivity modules push the last term and pull the speed term down.

The two numbers that matter. Belt throughput and machine output. A yellow belt moves 15 items per second, a red one 30, a blue one 45 — that is the ceiling a full belt can feed. If a build needs 30 items per second of something, you need a red belt (or two yellow) **and** enough machines to make 30/s. Worked example: an iron gear wheel takes 0.5 seconds in an assembler 2, so one assembler makes 1.5 gears per second; to fill a red belt of gears you need $30 \div 1.5 = 20$ assemblers. The same arithmetic runs backward: if 20 assemblers are consuming 30 plates per second, the plate belt feeding them must be at least red, and the smelters behind it must produce 30/s too.

The chain is the constraint. Ratios chain: miners feed furnaces, furnaces feed assemblers, assemblers feed science. Rather than hand-computing every link, the Wave 0 tools finish the job: **Factory Planner** computes the whole chain for a target throughput, and **Rate Calculator** measures what a selection of machines is actually doing. The Architect uses the formula to **understand**, the tools to **scale**.

Tip: Keep a handful of reference numbers in your head and the rest is division: 15/30/45 items per second for the three belts, 0.5/0.75/1.25 for the three assemblers, and 'a blue belt of X is 45 per second, which is the factory's real heartbeat'. When a line feels short, count the belt first — if the belt is saturated, add a lane; if it is not, the machines upstream are the bottleneck, and the P production screen shows which one stopped first.

Megabase Blocks: The Architect's Capstone

Everything in this chapter converges on the city block: a megabase built from identical, repeatable, self-contained cells, each served by rail and by its own robot network, orchestrated by circuits. This is where the Architect's mandate — 'good enough is a failure state' — actually pays off, because a block layout you design once is a layout you stamp a hundred times.

The block. A **city block** is a square of rail tracks — 100x100 tiles with a four-lane rail ring (two in, two out) around it, signals on every side, and a station (or pair of stations) inside. Inside the ring sits one production task: a smelting column, a circuit plant, a science cell. The block's station is its mouth — 'Iron In', 'Steel Out' — and its train limit is the throttle. Because every block is identical, expanding means stamping another block and naming its stops; the rail ring connects them all with no new design work.

The cells inside. Each block is split into a **belt-fed core** and a **robot-served fringe**. Heavy flows — plates off the train, bulk intermediates — move by belt within the block; high-mix logistics (repair packs, modules, science intermediates) are handled by a small local robot network. A requester warehouse feeds the assembly core; a provider warehouse swallows its output. The block is self-provisioning: it requests its inputs, provides its outputs, and the robot network settles the internal accounts — the **Warehousing Mod** and the whole logistics chapter working at scale.

The network glue. The blocks talk to each other by train and by circuit. A train limit on each station plus circuit-enabled stops (the enable-by-threshold build from the circuits chapter) keeps trains from flooding a full block, and Cybersyn can elevate that to fully automatic dispatching: providers and requesters negotiate by signal, and a pool of trains at a depot answers demand. On top of it all, the

Circuit HUD pins each block's key signals — buffer levels, station status, robot counts — to the screen, so a megabase of a hundred blocks is watched from one place instead of walked.

Tip: Design the block **once**, blueprint it, and treat it as frozen. The entire point of a city block is that you never redesign it: the rail ring is standard, the stations are standard, the internal lanes are standard, and the only thing that changes between blocks is which machines sit inside. When a block becomes a bottleneck, do not rebuild it — stamp a second block doing the same task and let the rail network and train limits share the load.

Warning: The megabase fails in three predictable ways, all of them train problems: junctions that cannot fit the longest train (deadlock), stations with no train limit (trains piling up on the main line), and a single giant robot network (robots flying across the whole map). The fixes are the ones this chapter already gave you — chain signals before every crossing, a limit of one train per station until proven, and small local bot networks — so treat those three as the checklist every block must pass before it is stamped.

The Architect's Roadmap

The whole wave, one glance.

1. **Reorganize onto the bus.** Lay a 4-iron, 4-copper, 2-green, 2-steel spine with dedicated pull-offs. This is **The Bus**.
2. **Make the rails a network.** One-way pairs, stop names, train limits, chain signals on every junction. This is **Trains**.
3. **Teach the factory to think.** Wire one real circuit — a chest, a decider, a train stop — then another. This is **Circuits**.
4. **Deploy the robot layer.** Low-mix logistics to bots, bulk to belts, a warehouse on each end of the busiest requests. This is **Logistics Robots**.
5. **Module the core.** Productivity in the machines, speed in the beacons, efficiency on the drills. This is **Modules & Beacons**.
6. **Bank some quality.** Run one small upcycling loop for legendary machines and modules; leave the rest normal. This is **Quality**.
7. **Do the math.** Pick a science goal and run the rate formula end to end — belts, assemblers, miners all counted. This is **Rate Mathematics**.
8. **Stamp the capstone.** Design one city block, blueprint it, and grow the megabase by copying it. This is **Megabase Blocks**.

Each step leaves a system that keeps working while you build the next one — the Architect's factory scales one honest layer at a time, and every layer you add now is one you will never have to tear down.

The Architect: Modlist

Wave 1 is where the factory stops being **good** and starts being **scalable**. The Castaway proved the rocket; The Architect has to prove the factory behind it — belt highways, a signalling rail network, circuit logic, robot logistics, modules and quality at volume. The mods in this chapter exist to deepen exactly those scaling systems, and none of them hands out power. Every card below passes the power-spike test: nothing raises an output cap, a throughput ceiling, an efficiency number or a cost for free. Where a mod genuinely increases what a belt or a building can carry, the increase is bought with research, materials, and a real logistics step the engineer has to build and pay for. If a mod ever started giving vanilla numbers away, it would fail this wave's one rule.

Twelve mods ship in Wave 1, split across the next three sections by what they change. **Mechanics** carries the four system mods — Project Cybersyn, Circuit HUD V2, Module Inserter and Auto Deconstruct — the train, circuit, module and expansion-lifecycle systems that drive the factory. **Content** carries the three logistics and production mods — Miniloader (Redux), Deadlock's Stacking Beltboxes & Compact Loaders, and Warehousing Mod — the new belts, loaders and containers the scaling factory is built from. **Graphics** carries the five visual mods — Nixie Tubes, Belt Visualizer, Prismatic Belts, Circuit Visualizer and Alien Biomes — the signal and overlay clarity that keeps a megabase readable, plus the terrain variety that gives the world it sits on its own geography. All twelve are verified on the official Factorio Mod Portal against Factorio 2.0.77; each card records the mod's last-update date, its 2.0-line build, and its dependencies, and every verdict is logged in STATUS.md.

The Architect: Mechanics

The four system mods in this section change **how the factory decides things**, not what it can produce. Cybersyn turns the rail network into a circuit-driven logistics network. Circuit HUD V2 puts the network's vital signals on the screen. Module Inserter hands the module slots over to machines and robots. Auto Deconstruct makes a depleted outpost flag itself for teardown. Each one deepens one of the scaling systems — trains, circuits, modules, expansion — and each one is a control or convenience change with no balance number moved. That is the wave's whole contract: the systems get smarter, the numbers stay vanilla.

Project Cybersyn - Logistics Train Dispatcher

Version/Updated: Updated 26 Jun 2026 (page); 2.0-line build v2.0.53 (15 Mar 2026). Newer 2.1-line releases (2.1.2, Jun 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Project Cybersyn - Logistics Train Dispatcher' matches the API title exactly.

Dependencies: None — standalone.

Impact: Turns trains into a logistics network. Place a cybernetic combinator beside a train stop and set it to 'station' or 'depot'; a positive circuit signal on the combinator means that stop *provides* that cargo, a negative signal means it *requests* it. A handful of trains parked at depots are then dispatched automatically to carry requested goods to requesting stations — the same provider/requester idea as robot chests, scaled to rail. Hand-written schedules give way to circuit-driven dispatching, which is exactly the 'circuit-linked logistics' this wave is built to teach.

Notes: By lesbian_mami; 56K+ downloads; designed for 2.0's train features and the actively maintained modern successor in spirit to the older LTN mod. Not a throughput buff and not a cheat — it automates *control*, and its demands are circuit signals, so it rewards the combinator skills the Strategy chapter teaches. Power-spike verdict: passes untouched.

Circuit HUD V2

Version/Updated: Updated 22 Jul 2026 (page); 2.0-line build v2.5.1 (24 Jun 2025). Newer 2.1-line releases (3.0.0, Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Circuit HUD V2' matches the API title exactly.

Dependencies: None — standalone.

Impact: Pins any circuit-network signal to the on-screen HUD — robot counts, chest stock, station buffers, fluid levels — so the vital signs of a whole megabase stay visible without opening

a single panel. It reads the circuit network and draws numbers; nothing about the network or the factory is modified. For the city-block era it is the missing instrument panel: wire a pole, pin a signal, and every block reports in.

Notes: By Xiomax; 12.5K downloads. Pairs naturally with Nixie Tubes (in-world readouts) and Circuit Visualizer (wiring clarity) — this is the always-on panel version of the same idea. Power-spike verdict: display only, passes untouched.

Module Inserter

Version/Updated: Updated 5 Feb 2026 (page); 2.0-line build v1.0.4 (5 Feb 2026) — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Module Inserter' matches the API title exactly.

Dependencies: None — standalone.

Impact: Lets inserters reach the module slots that are normally off-limits to them — the slots inside crafting machines and labs — so modules can be inserted, swapped and removed by belts, inserters and robots instead of by hand. As module-heavy production lines take over the mid-game, this is the difference between re-fitting a build by clicking a hundred machines and re-fitting it by stamping a blueprint with a requester chest of modules feeding the line. Pure QoL around the modules-and-beacons system: no module stat, recipe or cost is changed.

Notes: By protocol_1903; 17K downloads. Deferred from Wave 0 precisely because modules only matter from Wave 1 onward — this is where it earns its place. Re-verified fresh for this wave: v1.0.4 (5 Feb 2026), factorio_version 2.0, newest release on any line. Power-spike verdict: passes untouched.

Auto Deconstruct

Version/Updated: Updated 30 Jul 2026 (page); 2.0-line build v1.0.14 (30 Jul 2026). Newer 2.1-line releases (2.1-line build v1.1.2, 30 Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Auto Deconstruct' matches the API title exactly.

Dependencies: None — standalone.

Impact: Automatically marks a mining drill for deconstruction the instant its patch runs dry, so a spent outpost flags itself for the construction robots instead of sitting dead with four drills on empty ground. In a scaling factory, outposts are born and die continuously; this turns the teardown half of that lifecycle into the same hands-off process as the build. A marker helper only — it changes no recipe, cap or balance value.

Notes: By mindmix; 359K downloads, one of the portal's most-installed utilities. Deferred from Wave 0 because it belongs to the expansion-logistics theme this wave covers. Re-verified fresh for this wave: 2.0-line build v1.0.14 (30 Jul 2026). Power-spike verdict: passes untouched.

The Architect: Content

The three content mods in this section are the physical building blocks of the scaling factory: a loader that moves items onto and off belts in one tile, a belt-stacking system that multiplies what a single belt lane can carry, and a family of high-capacity warehouses that the logistic network can treat like giant

chests. All three are **new content** in the strict sense — new machines, new containers, new recipes — and all three obey the wave's gate: their benefits are bought with research, expensive materials, and a logistics step you build yourself. None of them hands vanilla numbers away.

Miniloader (Redux)

Version/Updated: Updated 4 Jul 2026 (page); 2.0-line build v1.1.0 (4 Jul 2026). Newer 2.1-line releases (2.1.0, Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Miniloader (Redux)' matches the API title exactly.

Dependencies: None — standalone.

Impact: A one-tile loader that loads and unloads *belts* — from a belt, onto a belt, or between a belt and a chest or wagon. Vanilla 2.0's loader only connects containers and wagons; Miniloader's whole point is the belt side, where a normal inserter is the only vanilla option. It comes in normal, fast and express tiers that follow the belt research line, supports sideloading from a belt, per-lane filters, and a speed mode for containers. It is advertised by its author as UPS-friendly, so a train-station or smelter line dense with loaders stays fast.

Notes: By hgschmie; 21K+ downloads; the actively maintained 2.0 continuation of the classic Miniloader. Power-spike verdict: no vanilla cap is raised — a miniloader's throughput is bounded by the belt or container it touches (an express belt still moves 45 items per second), and the tiers are gated behind the same logistics research as the belts they match. The 'chute loader' option that would hand belt-loading out early is off by default.

Deadlock's Stacking Beltboxes & Compact Loaders

Version/Updated: Updated 6 Jan 2025 (page); 2.0-line build v2.6.0 (6 Jan 2025) — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Deadlock's Stacking Beltboxes & Compact Loaders' matches the API title exactly.

Dependencies: None — standalone (the Bob's/Factorio Extended integrations live in a separate mod).

Impact: Lets you *stack* up to five copies of certain items — ores, plates and select intermediates — into a single stacked item that rides the belt, multiplying that belt lane's effective capacity fivefold. A stacking beltbox at the start of a line compresses the flow; a beltbox at the destination unstacks it back into normal items machines can eat. The mod also bundles its own compact 1x1 loaders. This is Wave 1's 'new belts and containers' content: throughput rises only where you build the beltboxes and handle the stack-and-unstack step.

Notes: Originally by Deadlock989, maintained by shanemadden (the portal owner); 126K+ downloads — the canonical belt-stacking mod. *Borderline power-spike verdict:* 5x compression is a real throughput raise above vanilla belt caps, but it is bought with dedicated beltbox buildings, their material cost, and an explicit unstacking step before any machine can consume the goods — a proportionate cost. ACCEPTED per the written verdict in STATUS.md.

Warehousing Mod

Version/Updated: Updated 10 Dec 2024 (page); 2.0-line build v1.0.3 (10 Dec 2024) — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Warehousing Mod' matches the API title exactly.

Dependencies: None — standalone.

Impact: Adds 6x6 warehouses and cheaper 3x3 storehouses, each available in the whole logistics chest family — passive provider, active provider, storage, requester and buffer — so the robot network can treat a warehouse as one enormous chest that requests and provides. Red and green wires attach at the corners, so a warehouse's contents can be read and controlled by the circuit network. Storage expansion only: no recipe, throughput or efficiency number changes, and the logistic versions inherit the same research gates as their chest equivalents.

Notes: By anoyomouse; 236K+ downloads, one of the most-installed logistics mods on the portal. v1.0.3 (Dec 2024) is its newest release on any line and covers the whole 2.0.x line; the mod is stable and long-shipped. The gate is material cost — warehouses are deliberately expensive, steel-heavy buildings, and the requester versions sit behind the vanilla requester-chest research.

The Architect: Graphics

The five visual mods in this section make the scaling factory **readable**. A megabase fails on information long before it fails on production: which belt is this, which way does the wire flow, is this tank full, where does the yellow line actually go. Two of these mods turn circuit signals into something you can see — glowing tube digits and pinned HUD numbers — and two of them light up the factory's arteries — belt lines and circuit wires — so the answer is a glance instead of a chase. The fifth, Alien Biomes, varies the terrain and trees the factory is carved from, so the world outside the bus stays as distinct as the factory on it. None of them changes a single game value.

Nixie Tubes

Version/Updated: Updated 23 Jun 2026 (page); 2.0-line build v2.0.9 (7 Nov 2025). Newer 2.1-line releases (2.1.0, Jun 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Nixie Tubes' matches the API title exactly.

Dependencies: None — standalone.

Impact: Adds nixie-tube displays that connect to a circuit network and show the current value of a signal as glowing digits — storage levels, station buffers, robot counts, anything you can wire. Instead of hovering a chest or opening a panel, you read the number off the wall of the factory. It is a pure signal display: the circuit network and everything it controls behave exactly as before.

Notes: By justarandomgeek; 98K+ downloads, the classic circuit-display mod. It lands in graphics because its job in this wave is making circuit values visible at a glance, alongside Circuit HUD V2's always-on panel. Power-spike verdict: display only, passes untouched.

Belt Visualizer

Version/Updated: Updated 7 Jul 2026 (page); 2.0-line build v2.0.2 (22 Oct 2024). Newer 2.1-line releases (2.1.4, Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Belt Visualizer' matches the API title exactly.

Dependencies: None — standalone.

Impact: Press a hotkey while pointing at any belt and the entire connected line lights up; press again and it cycles through the lanes of that line. Tracing where a belt actually goes — and finding the single empty stretch that is starving a whole block — becomes a glance instead of a long walk. A visual overlay only; no belt is altered.

Notes: By _CodeGreen, the same author as Wave O's Squeak Through 2; 125K downloads. Its 2.0-line build (2.0.2, 22 Oct 2024) covers the whole 2.0.x line, and the author keeps shipping 2.1 updates. Power-spike verdict: passes untouched.

Prismatic Belts

Version/Updated: Updated 20 Jul 2026 (page); 2.0-line build v3.0.4 (26 Jun 2026). Newer 2.1-line releases (3.1.3, Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Prismatic Belts' matches the API title exactly.

Dependencies: None — standalone.

Impact: Colors each belt's conveyor surface to match its direction arrows, so the way every belt runs is visible at a glance — which matters most on a crowded main bus and inside dense city blocks where four parallel belts can otherwise blur together. Pure visual clarity; belt behaviour is untouched. Per the author's description it also supports the loaders added by Wave 1's content mods — Miniloader and Deadlock's Stacking Beltboxes & Loaders — so those stay color-matched too.

Notes: By Kirazy; 20.6K downloads. A small, configurable tweak that pays off more the bigger the factory gets. Power-spike verdict: passes untouched.

Circuit Visualizer

Version/Updated: Updated 17 Nov 2024 (page); 2.0-line build v2.0.2 (17 Nov 2024) — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Circuit Visualizer' matches the API title exactly.

Dependencies: None — standalone.

Impact: Overlays every circuit wire with the signals currently flowing on it, turning a tangle of combinators into a readable diagram of values. When a decider is mis-wired or a signal is crossing where you thought it wasn't, the answer is drawn on the screen instead of guessed at. Read-only visualization of the circuit network; no signal or entity is changed.

Notes: By _1024; 21.5K downloads. The circuit-network companion to Belt Visualizer — one lights up belts, this lights up wires. v2.0.2 (Nov 2024) is the latest release on any line and is stable across the 2.0.x line. Power-spike verdict: passes untouched.

Alien Biomes

Version/Updated: Updated 24 Jun 2026 (page); 2.0-line build v0.7.4 (23 Jun 2025).

Verified: YES — portal page + API fetched 2026-08-03; page title 'Alien Biomes' matches the API title.

Dependencies: None — standalone.

Impact: Replaces Nauvis's single grass-and-desert palette with a far more varied terrain — snow, volcanic, crater and many dirt, sand and grass variants — plus new tree and decorative types. Where the four signal mods make the factory readable, this makes the *world* readable: distinct biomes and tree belts give a sprawling city block a sense of place and make territory boundaries obvious. Environment and terrain visuals only: no ores, recipes or balance numbers are touched.

Notes: By Earendel; 700K+ downloads, one of the portal's most-installed visual mods. Moved into this wave from Wave 0 during guide revision (2026-08-04). Terrain generation is locked at world creation, so enable it before starting The Architect's new game; it will not retroactively re-texture an existing save. The 2.0.77 install uses v0.7.4. Power-spike verdict: passes untouched.

Wave 2: The Krastorio Prodigy

The Krastorio Prodigy: Strategy

The Castaway built a rocket. The Architect built a megabase. The Prodigy has to rebuild **everything**. Krastorio 2 is an overhaul “focusing on end-game technologies and moderately increased complexity” — which understates it: more than 40 new buildings, more than 100 new technologies, three new resources, a research system rebuilt around tech cards, and a combat model where bullets are physical objects. On top of that, this wave enables Space Age and bridges it in with Krastorio 2 Spaced Out, which merges the two overhauls into one longer, more complex playthrough. Nothing you memorised in Waves 0 and 1 transfers as fact — the ratios are different, the science packs are gone, and even the win condition has moved. What transfers is the **method**: read the data, plan the chain, build the scaling systems, and let the tools do the arithmetic. This chapter tells you what Krastorio 2 changes and how the Prodigy survives it. Every structural claim below is sourced from the mods’ own portal descriptions and changelogs; specific recipe numbers are deliberately not asserted, because the right way to learn a rebalanced recipe is to look it up in-game (FNEI), not to trust a guide’s memory.

The New Ores: Imersite, Mineral Water, Rare Metals

Krastorio 2 adds three resources, and its bridge with Space Age keeps them on Nauvis where the base game’s engineers already live. They are not decorative — each one is a new branch of the production tree.

Imersite is the star. Krastorio 2’s page describes it as “a rare end-game resource used in many advanced recipes” — useless on its own, but “when used in advanced manufacturing processes, exhibits ludicrous tensile strength and superconductive abilities.” In practice that means imersite is the material the end-game is built from: the interesting question is always what advanced process consumes it next. In this install the **Krastorio 2 Imersite Asteroids** mod (Content chapter) makes imersite a **space-first** resource: it locks the Nauvis imersite drill behind the advanced tech card, so until that research you bootstrap imersite from space — imersite asteroids en route to Aquilo and in the belt between Aquilo and the inner planets, with trace amounts in Fulgora orbit — and ground mining unlocks later as the second source.

Mineral water is “water impregnated with various silts and dusts from the planet’s surface” and is used primarily in the production of lithium. Where vanilla has one water source, K2 has two: plain water for boilers and chemical processes, and mineral water for the lithium branch that feeds batteries and beyond. Pump it like water — it behaves like a fluid patch — but treat it as a distinct supply line, because lithium production depends on it and lithium is a common ingredient once the mid-game starts.

Rare metals are “an analogous representation of various rare elements (i.e. titanium, platinum, gold)” — a mined metal resource that behaves like iron or copper ore but appears in smaller, more valuable patches and feeds a tier of recipes the common metals cannot. Where iron and copper are your volume, rare metals are your **quality** — expect them to feed advanced circuits, batteries, armour and late machines.

Tip: YARM from the Wave 0 base tracks any resource patch, and K2’s new ores are patches like any other — set a YARM watch on your nearest rare-metals patch the moment you find one, because it depletes like a metal and feeds the whole mid-game. Factory Planner and Rate Calculator see K2’s recipes exactly as the game does, so the moment a new ore appears, plan the chain before you build it.

Warning: Do not let the names fool you into vanilla habits. Mineral water is **not** a substitute for normal water — a base that routes mineral water into its boilers is a base that stalls its power when the lithium line needs every drop. And imersite is explicitly useless on its own — and, in this install, gated: the ground drill stays locked until the advanced tech card, so bank whatever imersite the asteroids give you rather than building a production line around it before the advanced recipes unlock. Stockpile it, and let the tech tree tell you when it matters.

New Intermediates & Processing Chains

Every new resource feeds a chain of new intermediates, and K2's overhaul multiplies the number of **steps** between raw materials and finished goods. The lithium produced from mineral water is itself an intermediate; rare metals become specialised products; imersite spawns a family of advanced materials used in end-game machines and buildings. On top of the new resources, K2 adds "more granular fuel trade-offs" and radioactivity, which push the fuel and power chains in directions vanilla never explored.

The Prodigy's discipline in this wave is simple: **never guess a chain**. The base game's recipe set is small enough to memorise; K2's is deliberately not. FNEI (this wave's recipe browser) answers "what makes this?" and "where is this used?" for every item in the game, and it reads the live game data, so it is always right even after a rebalance. The workflow that keeps a K2 base honest is the same one Wave 0 taught, just used more: FNEI to learn what an item needs, Factory Planner to design the block that makes it at the right rate, Rate Calculator to confirm the real machines are hitting the plan, and the bus or rail spine from Wave 1 to move it.

Tip: Bind FNEI to a comfortable key and treat it as part of the HUD. A rebalanced recipe is not a puzzle to solve from memory — it is data to look up. Every time a chain stalls because 'the recipe must have changed', FNEI is the two-second answer.

Warning: Do not build a production block for an item you have not looked up. K2's recipe rebalances mean an intermediate you remember costing one thing now costs another, and a block built on a stale recipe is a block that silently under-produces forever. Check the recipe when you design it, and re-check it the first time the block runs short.

Research Overhaul: Tech Cards & Research Computers

This is the single biggest conceptual change in the wave. Krastorio 2's page states it plainly: "Instead of science packs, you produce tech cards that are inserted into research computers." The vanilla science-pack ladder — red, green, blue, military, production, utility — is gone. In its place:

Tech cards are the new research currency, produced by assembling machines and carried to the lab line like science packs used to be. But they are fed into **research computers**, K2's replacement for the vanilla lab, and there are **three tiers of tech card**, each of which "require their own laboratories." So a K2 research setup is not one lab type fed by six packs — it is (at least) three lab lines, each tied to its own tier of tech card, unlocked and expanded in sequence.

The second half of the overhaul is the **phase-out**. The page notes that "early-game tech cards will be phased out as the game progresses, allowing you to redirect resources to other parts of your factory." A tier-one tech card that powers the early game stops being needed once you can make higher tiers — the early card production line gets dismantled and its machines go back to the factory. This is K2's way of keeping the early research economy lean: progress literally frees up capacity instead of accumulating demand forever.

Under the Krastorio 2 Spaced Out bridge, the tech-card ladder extends into space. K2SO's page describes the higher tiers directly: **advanced tech cards** unlock powerful new buildings on the Space Age planets (an advanced furnace on Vulcanus, laser artillery on Fulgora, an advanced chemical lab on Gleba, and more); **matter tech cards**, made from samples of every resource, enable material transmutation — including of Space Age resources; and **singularity tech cards**, space-exclusive science made of pure matter and asteroid chunks, unlock instant interplanetary travel and the highest tier of logistics. K2SO also adds two new labs, including a space-exclusive singularity lab, and its changelog records that the promethium tech card must be researched in space using singularity labs.

Tip: The phase-out is the research economy's gift — when a tech-card tier becomes obsolete, dismantling its production returns a whole factory's worth of machines. Plan for it from the start: give each early card line a clean footprint you are willing to lose, and do not grieve over a card tier you outgrow. That freed capacity is the wave's own refund for progress.

Warning: Do not build one giant 'science block'. The three tiers each need their own laboratories and their own supply, and the tiers unlock at different times — a single mixed line will either stall the early tiers or starve the late ones. Build per-tier labs from the moment a tier exists, and let the phase-out retire the oldest line rather than the newest. Under K2SO, remember the high tiers are **space** science: singularity tech cards are made in space, in the singularity lab, and the promethium science they gate must also be researched in space.

Adapting the Vanilla Systems

K2 does not add a layer on top of vanilla — it rebalances the base systems themselves, and the Prodigy adapts each one. The general rule: **expect every ratio to differ, and verify before you build**. Specific numbers are deliberately not quoted here, because K2's rebalances are exactly the kind of fact that changes between releases; the tools — FNEI, Factory Planner, Rate Calculator — are the reliable source.

Smelting. K2 changes the metal economy around the new resources and rebalances furnace recipes and costs — expect to meet new buildings and technologies across the smelting line as the wave progresses. The iron-and-copper columns you built in earlier waves will not produce at vanilla ratios. Adapt by re-running the plan: treat a furnace column as a black box whose input-to-output number you re-derive with Factory Planner instead of trusting memory.

Oil and fluids. K2's recipe rebalances push fluid volumes up, and its "more granular fuel trade-offs" make the fuel economy a real design problem. The pipe network becomes a first-class scaling constraint — which is exactly why the Krastorio team recommends **Fluid Must Flow** (this wave's Mechanics chapter): very large pipes with enormous throughput turn a wall of parallel pipes back into a single, buildable trunk. Adapt by thinking about fluid capacity the way Wave 1 taught you to think about belt capacity, and by treating mineral water as a separate fluid line from ordinary water.

Power. K2's feature list includes radioactivity and more granular fuel trade-offs, so the vanilla coal-to-steam ladder is only the start; expect the fuel and power economy to climb through tiers, with new energy sources along the way. Adapt by building power headroom **before** you need it — a rebalanced factory consumes differently than the one in your head, and the P screen plus a small circuit alarm (Wave 1 skills) will catch a power cliff before the inserters stall.

Trains. Trains stay the bulk-hauling backbone — and K2 adds new buildings, vehicles and equipment, with "almost all vehicles now have equipment grids." That last change is a quiet QoL upgrade: vehicles become carriers for equipment in a way vanilla limits. Adapt by keeping the Wave 1 rail habits (train limits, chain signals, one-way pairs) — none of them change under K2 — and by treating

the new ore patches as normal outposts with normal train schedules, which is exactly what the new resources demand.

Robots. The logistic network works as it always did, but K2 brings its own roboports and logistics buildings. The **Krastorio2 - roboport mode colored textures** mod (Graphics chapter) makes the new roboports readable by tinting their rings to their mode. Adapt by keeping the Wave 1 rule — belts for bulk, bots for high-mix — and by learning K2’s roboport modes, which the textures exist to make obvious.

Tip: The single most reliable adaptation tool in this wave is the Wave 0 planner stack. Factory Planner does not need to know K2’s recipe set by heart — it reads it from the game. Whenever a vanilla habit starts to feel wrong, that is not the mod fighting you; it is the rebalance working as intended. Re-derive the plan and move on.

Warning: The biggest adaptation failure in an overhaul wave is **assuming**. Assume the old ratio, assume the old recipe, assume the old power curve — and build a block that is quietly wrong. The Prodigy never assumes: every block is planned from current data, and every plan is verified against the real machines with Rate Calculator before it is trusted.

Combat & Biters: Ballistics Against Snappers

Krastorio 2’s combat overhaul is not a buff — it is a **physics change**. The page is explicit: “Bullets are now physical objects that have speed, wind resistance, and various AoE effects. Bullets have to travel through space to their target, and can be blocked by solid objects.” Two consequences change the whole defence game. First, a shot can **miss** — a projectile has to travel to the target and can be blocked by solid objects in between, so a wall of turrets behind a solid wall can find its own wall in the way. Second, **auto-aim is disabled by default**, requiring the player to choose what to aim at; K2 includes a startup setting to re-enable auto-aim for players who prefer it (or for multiplayer, where the manual model does not play nicely with latency).

On top of the physics, this wave ships **Armoured Biters** (Content chapter), one of the K2 team’s recommended companions: a new enemy, the Snappers — “heavily armoured but slower than regular biters.” Put the two together and the early defence loop changes shape: the first turrets may not reliably hit, and the armour soaks what does hit. The Prodigy’s answer is not bigger guns alone; it is **geometry** — clear fields of fire with nothing solid between muzzle and target — and **volume** — enough turrets that even a low hit rate and soaked damage still converge. K2’s own weapons and armour tiers grow with the wave, and the armour/equipment line (higher-tier equipment, equipment grids on almost all vehicles) gives the engineer the survivability the harder enemies demand.

Tip: With projectiles that can be blocked by solid objects, line-of-sight is now a real mechanic: keep a clear lane between your gun turrets and the field they defend, and never stack turrets so the first row blocks the second. Start with the startup setting for auto-aim left **on** if the manual model frustrates you — it exists precisely so the combat overhaul does not become a control fight — and turn it off once you trust your aim.

Warning: Kiting biters in a circle — the reliable vanilla cheese — stops being reliable when enemies are armoured and projectiles can miss. Armoured Biters is a difficulty-raising companion by design: it exists so the wave’s combat asks for actual defence, not just a fast pair of boots. Respect the wall-and-turret economy the same way Wave 0 taught, and expand defence before the pollution cloud grows, because the Snappers are the least of what the end-game sends.

The Endgame: Deep Science and the Win

The Prodigy's victory is not a rocket — it is a **deep science push** through the whole rebuilt tech tree to Krastorio 2's intergalactic transceiver, and under the Space Age bridge the road is longer still. The base game's first launch no longer wins the day; the win condition in this wave is the end of K2's tech tree, reached only after the tech-card economy has climbed all three tiers (and, under K2SO, the advanced, matter and singularity tiers).

Under Krastorio 2 Spaced Out, the endgame changes shape in three concrete ways. First, the **intergalactic transceiver** — K2's win building — is itself reworked: K2SO's changelog records that it must now be made in space and that its recipe was reworked, so the final construction is a space-platform project, not a Nauvis one. Second, **promethium science becomes mandatory for the win condition** and unlocks the ultimate power source — the bridge's changelog confirms the promethium tech card must be researched in space using the singularity lab. Third, the **singularity tech cards** — space-exclusive science made of pure matter and asteroid chunks — gate instant interplanetary travel and the highest tier of logistics, so the very end of the run is about logistics in space as much as it is about science on the ground.

Imersite sits at the heart of this push: the end-game machines and advanced recipes are exactly where the “ludicrous tensile strength” material is consumed. With the **Krastorio 2 Imersite Asteroids** mod the supply is a **bootstrap** problem before it is an endgame one — the Nauvis drill is locked behind the advanced tech card, so imersite must come from space (asteroids en route to Aquilo and in the belt between Aquilo and the inner planets, plus trace amounts in Fulgora orbit) until ground mining unlocks. The Prodigy's endgame plan is therefore three-legged: keep the tech-card economy fed through every tier, build the space platform and logistics to assemble the transceiver in orbit, and keep a steady imersite supply reaching the advanced machines that turn it into the end-game materials.

Tip: Treat the endgame as a logistics exercise first. The deep-science push consumes enormous quantities of late tech cards, and under K2SO much of the final construction happens in space — so the machine that finishes the run is the logistics network (rail, belts, and the train-dispatch habits Wave 1's Cybersyn chapter taught, scaled up). Nail the imersite supply line early: bank it while it is cheap, because the end-game machines consume it by the belt-load and late-game scarcity is the classic Prodigy stall.

Warning: Do not let the base game's rocket timing fool you. In this wave the first rocket — or even the first few — are milestones, not the win; the transceiver, promethium science and the singularity research all sit well past it. Budget for a long tail: the deep-science push will outlast everything you built in Waves 0 and 1, and that is the point of the Prodigy's name. Milestones (Graphics chapter) will show you the curve — use it to pace a run that is measured in dozens of hours, not one launch.

The Prodigy's Arc

The three waves were always one story. The Castaway is the castaway — a lone engineer who proves a rocket can be built. The Architect is the builder — the engineer who proves a factory can scale into a megabase of blocks, rail, circuits and robots. **The Krastorio Prodigy** is the master — the engineer who proves they can take a factory where every rule is rewritten and still win. The arc is deliberately about **mastery of change**: nothing from the first two waves transfers as a fact, but everything transfers as a skill. The bus, the rail network, the circuit logic, the robot economy, the module math

— all of it still works, all of it still earns its keep, and all of it now has to be **re-verified** against a rebalanced world.

That is the wave's theme, and its mods support it thematically as well as mechanically. **FNEI** is the scholar's tool — the way the Prodigy learns a rebalanced world from its own data. **Milestones** is the historian's ledger — the record of how fast the ladder was climbed, which turns the whole wave into a scored journey with a history worth looking back on. **Armoured Biters** and **Fluid Must Flow** are the K2 team's own nods that the master's world is bigger — harder enemies, more demanding fluids — and **Imersite Asteroids** and the **tech card icons** tie the two anchors together into one coherent space. The RP capstone is simple: by the end of this wave you are no longer the engineer who survived, or the one who scaled — you are the prodigy who **rebuilt**, and who finished a factory that two overhauls conspired to make hard.

Tip: Let the wave's tools tell its story. Start Milestones on day one of the new save and watch the ladder form as the tech cards phase in. Use FNEI the moment the first new recipe confuses you. When the intergalactic transceiver finally assembles in space, open the Milestones history and look back — the wave's whole point is that you can see how far the Prodigy travelled.

Warning: The Prodigy's failure mode is hubris — assuming the old rules still hold and building at scale before re-verifying. The wave punishes that with stalled chains, starved fluids, and a defence that cannot hit. Stay humble at the beginning of each new system: plan from data, verify with tools, and expand the scaling layers from Wave 1 only once the rebalanced facts are confirmed.

The Krastorio Prodigy's Roadmap

The whole wave, one glance.

1. **Start the K2SO save.** Space Age enabled, Krastorio 2 + Krastorio 2 Spaced Out at the top, the Wave 0 QoL base carried in. Start Milestones on day one. This is **The Setup**.
2. **Learn the new ground rules.** Read FNEI for the first rebalanced recipes; note the mineral water, rare metals and imersite patches. This is **The New Ores**.
3. **Build the first tech-card economy.** Research computers, tier-one labs, the first tech cards — and plan the line so the phase-out can retire it cleanly. This is **Research Overhaul**.
4. **Re-derive the vanilla systems.** Rebuild smelting, fluids, power and the fuel economy against current data; bring in Fluid Must Flow for the pipe trunk. This is **Adapting the Vanilla Systems**.
5. **Make the factory scale again.** Rebuild the bus and rail spine from Wave 1 around the new resources; YARM watches the new ore patches, and the train-dispatch and circuit skills Wave 1 taught do the scaling work. This is **Scaling the Rebalanced Factory**.
6. **Respect the new combat.** Clear fields of fire, volume of turrets, Armoured Biters respected, auto-aim set to taste. This is **Combat & Biters**.
7. **Climb the card tiers.** Tier two, then tier three; dismantle each retired card line as the phase-out dictates. This is **The Tech-Card Ladder**.
8. **Go to space.** Advanced, matter and singularity tech cards; imersite asteroids in the belt; the singularity lab in orbit. This is **The K2SO Bridge**.
9. **Assemble the win.** Build the intergalactic transceiver in space, research the promethium science, unlock the ultimate power source, and push the deep science to its end. This is **The Endgame**.
10. **Look back.** Open Milestones, review the ladder, and close the Prodigy's arc. This is **The Capstone**.

Each step leaves the factory working while you build the next — and every step is a re-verification, a chance to prove the old skills still hold in a world where every rule changed. The Prodigy’s reward is the one thing the first two waves could not give: the knowledge that it was done **twice** — once by the numbers, and once with the whole book rewritten.

The Krastorio Prodigy: Modlist

Wave 2 is the capstone, and it plays by a different rule than the first two waves. The Castaway proved a factory can be built and launched a rocket; The Architect proved it can scale. **The Krastorio Prodigy** proves the engineer can master a factory that is **rebuilt** — every recipe rebalanced, every science tree replaced, combat turned into ballistics, and the win pushed to the very edge of the tech tree. This is the wave where **Space Age stays enabled**, and where the two anchors do the heavy lifting: **Krastorio 2**, the end-game-focused overhaul, and **Krastorio 2 Spaced Out**, the bridge that makes K2 run correctly with Space Age (K2’s own page is explicit that without the bridge, “the game will load, but progression will be broken”). Together they form the most demanding and most rewarding setup this guide covers.

Nine mods ship in Wave 2. The wave’s one rule is compatibility: every mod must be verified against the 2.0.77 target **and** against both anchors — K2’s rebalanced recipes and the K2SO bridge. The strongest evidence in the whole wave belongs to three of the four officially recommended companions on the Krastorio 2 page, which ship here alongside the two overhauls. **Content** carries the four overhaul and content mods — Krastorio 2, Krastorio 2 Spaced Out, Krastorio 2 Imersite Asteroids and Armoured Biters. **Mechanics** carries the two system mods — FNEI and Fluid Must Flow. **Graphics** carries the three visual mods — Milestones, K2 Tech Card Icons for Space Age, and Krastorio2 - roboport mode colored textures. All nine are verified on the official Factorio Mod Portal; each card records its last-update date, its 2.0-line build, and its dependencies, and every verdict is logged in STATUS.md.

The Wave 0 QoL base carries into this wave. Even Distribution, Squeak Through 2, Rate Calculator, Factory Planner, YARM, Todo List and Bottleneck Lite were each re-verified against K2 for this chapter: all seven are pure client, UI or informational mods that cannot touch K2’s rebalanced recipes (Rate Calculator and Factory Planner read the game’s real data, so they see K2’s recipes exactly as the game does; YARM monitors any resource patch including the new ores). Each carry-in verdict is logged in STATUS.md. The Wave 1 content and system mods — Miniloader, Deadlock’s beltboxes, Warehousing, Cybersyn, Alien Biomes and the rest — deliberately do **not** carry: the content mods are recipe-adjacent and K2 rebalances every recipe, and this wave keeps its install deliberately focused on the two overhauls, their verified companions, and the carried QoL base rather than the Wave 1 scaling stack.

The Krastorio Prodigy: Mechanics

The two system mods in this section exist because Krastorio 2 makes the engineer’s information and fluid problems **bigger** before it makes them harder to solve. K2 rebalances every recipe and adds hundreds of new ones, so the recipe browser that was merely useful in the base game becomes the wave’s primary reference tool. And K2’s rebalanced recipes stress fluid throughput in ways the base game never does, which is exactly why the Krastorio team ships its own fluid companion. Neither mod moves a number the player relies on: one reads the game’s data, the other adds new buildings that the engineer must research and build. Both are verified against the 2.0.77 target and against both wave anchors — the K2 team’s own recommendations carry the strongest compatibility evidence this wave has.

FNEI

Version/Updated: Updated 11 Jul 2026 (page, 2.1-line v0.4.7); 2.0-line build v0.4.6 (30 Mar 2025), factorio_version 2.0.

Verified: YES — portal page + API re-fetched 2026-08-03; page title 'FNEI' matches the API title exactly. Re-verified fresh from mod-ideas.md, where it was deferred to this wave for exactly this reason.

Dependencies: None — standalone, and pure client-side: it reads the game's recipe data, so it works with any overhaul including Krastorio 2.

Impact: The recipe and usage browser: for any item, it lists every recipe that produces it and every recipe that consumes it, plus the relevant machines and fuel values. In K2's overhaul — hundreds of new recipes, new resources, a rebuilt research tree — FNEI is the difference between guessing a chain from memory and tracing it from the data. Where vanilla players could learn the handful of vanilla chains, a K2 engineer leans on FNEI constantly to answer 'what makes this?' and 'where is this used?'.

Notes: By npo6ka; 357K downloads. Deferred from Wave 0 and Wave 1 precisely because its value peaks in an overhaul wave. Complements the Wave 0 tools: Factory Planner plans the chain, Rate Calculator measures it, FNEI explains what anything in it is. Power-spike verdict: informational only, passes untouched.

Fluid Must Flow

Version/Updated: Updated 25 Jun 2026 (page, 2.1-line v1.5.0); 2.0-line build v1.4.4 (22 Sep 2025), factorio_version 2.0.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Fluid Must Flow' matches the API title exactly.

Dependencies: None — standalone. No Krastorio 2 dependency is declared, but the Krastorio 2 page lists Fluid Must Flow under its 'Recommended companion mods', and it is by the Krastorio team (author raiguard, K2's maintainer) — the strongest compatibility evidence available.

Impact: Adds very large pipes with enormous throughput capabilities, giving a scaling factory a way to move oil, gas and water by the pipe-load instead of by walls of parallel pipes. K2's rebalanced recipes push fluid volumes well past what base-game pipe mechanics handle comfortably; this is the Krastorio team's own answer to that demand.

Notes: By raiguard; 244K downloads. *Borderline power-spike verdict (written in STATUS.md):* it raises the effective throughput a pipe can carry above vanilla caps, but the new pipes are separate buildings with their own recipes, gated behind research, and the mod exists specifically to serve K2's increased fluid demand — the K2 team recommends it in the K2 page's companion list. ACCEPTED.

The Krastorio Prodigy: Content

Wave 2 is anchored by the two overhauls in this section: **Krastorio 2** — an end-game-focused overhaul of nearly every system in the game — and **Krastorio 2 Spaced Out**, the bridge that lets Krastorio 2 run correctly with the Space Age expansion, which is enabled for this wave. They ship first, as a pair: Krastorio 2's own page states that it does not properly support Space Age on its own ("the game will load, but progression will be broken") and that the bridge add-on is required, and the bridge in turn requires Krastorio 2. Every other mod in this wave was checked against **both**

anchors before it earned a card. Two more content mods complete the section: one that turns K2's signature end-game resource into a space resource for the K2SO bridge, and one of Krastorio 2's officially recommended companions that makes the wave's combat genuinely dangerous. All four cards are verified on the official Factorio Mod Portal against Factorio 2.0.77, with each verdict logged in STATUS.md.

Krastorio 2

Version/Updated: Updated 29 Jun 2026 (page, 2.1-line v2.1.2); 2.0-line build v2.0.19 (26 Jun 2026), factorio_version 2.0. Newer 2.1-line releases (2.1.x, Jun 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API re-fetched 2026-08-03; page title 'Krastorio 2' matches the API title exactly. Re-confirms the original Task 3 verdict (2.0.19, factorio_version 2.0, COMPATIBLE).

Dependencies: Requires Factorio 2.0 base plus its own libraries (flib, Krastorio2Assets, Krastorio2MenuSimulations) which the built-in mod manager installs automatically. To run alongside Space Age it requires the Krastorio 2 Spaced Out bridge below — the K2 page states that without the bridge 'the game will load, but progression will be broken'.

Impact: The anchor overhaul, focused on 'end-game technologies and moderately increased complexity'. It adds over 40 new buildings, over 100 new technologies and more than 300 new unique icons; three new resources (imersite, mineral water, rare metals); a research overhaul built on tech cards and research computers; a full combat overhaul in which bullets become physical projectiles; radioactivity; more granular fuel trade-offs; and equipment grids on almost all vehicles.

Notes: By raiguard (originally created by Krastor and Linver); 385K+ downloads — the most-installed total overhaul on the portal. Everything else in this wave is verified against K2's rebalanced recipes; nothing in the wave restores the vanilla behaviour K2 intentionally replaced.

Krastorio 2 Spaced Out

Version/Updated: Updated 28 Jul 2026 (page, 2.1-line v2.0.14); 2.0-line build v1.6.21 (7 Jul 2026), factorio_version 2.0, feature_flags ["space-travel"]. Newer 2.1-line releases (2.0.x, Jul 2026) are outside the 2.0.77 target.

Verified: YES — portal page + API re-fetched 2026-08-03; page title 'Krastorio 2 Spaced Out' matches the API title exactly. Re-confirms the original Task 3 verdict (1.6.21, factorio_version 2.0, space-travel feature flag, COMPATIBLE).

Dependencies: Requires Krastorio 2, the Space Age expansion (the space-travel feature flag; owned), and its own asset pack (k2so-assets, auto-installed). Declared as a conflicting dependency: factorioplus.

Impact: The bridge that merges Krastorio 2 with Space Age, and the wave's second anchor. Per its page: it adds K2's resources (rare metals, mineral water, imersite) to Nauvis, remixes the mid-game planets, expands the research tree around collecting and processing planetary data, adds advanced tech cards that unlock new buildings on Space Age planets (advanced furnace on Vulcanus, laser artillery on Fulgora, advanced chemical lab on Gleba, and more), matter tech cards that enable material transmutation, and singularity tech cards — space-exclusive science made of pure matter and asteroid chunks — plus two new labs including a space-exclusive singularity lab. Promethium science becomes mandatory for the win condition.

Notes: By Polka_37; 53K+ downloads. The page notes development has moved to Factorio 2.1; the 2.0 line is stable at v1.6.21 for a 2.0.77 install. K2SO's changelog records that the intergalactic transceiver must be made in space under the bridge.

Krastorio 2 Imersite Asteroids

Version/Updated: Updated 30 Jun 2026 (page, 2.1-line v1.3.0); 2.0-line build v1.2.2 (14 Jun 2026), factorio_version 2.0.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Krastorio 2 Imersite Asteroids' matches the API title exactly.

Dependencies: Requires Krastorio 2 Spaced Out (declared dependency on the portal).
Optional: Asteroid Belt.

Impact: Raises the difficulty of K2's signature resource by *postponing* ground imersite mining: the mod locks the Nauvis imersite drill behind the advanced tech card, so until that late research you cannot mine imersite on Nauvis at all. Instead you must bootstrap it from space — the mod adds an imersite asteroid type encountered en route to Aquilo and in the belt between Aquilo and the inner planets (a pre-Aquilo technology), with trace amounts also in Fulgora orbit. Ground mining becomes the later, second source once the advanced tech card unlocks. K2SO's own page describes the effect as pushing imersite mining later, toward Aquilo.

Notes: By snakeru; 6.4K downloads. Recommended by the K2SO author on the Krastorio 2 Spaced Out page. Power-spike verdict: a *challenge-raiser*, not a handout — it delays a resource that would otherwise be ground-mined early and forces a space bootstrap, so it passes the power-spike test cleanly (it gives nothing away; it takes ground access away and gates it behind advanced research).

Armoured Biters

Version/Updated: Updated 7 Apr 2026 (page); 2.0-line build v2.0.2 (7 Apr 2026), factorio_version 2.0 — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Armoured Biters' matches the API title exactly.

Dependencies: None — standalone (optional dependencies: Space Age, Alien Biomes). No Krastorio 2 dependency is declared, but the Krastorio 2 page lists Armoured Biters under its 'Recommended companion mods' — the K2 team's own compatibility confirmation.

Impact: Adds a new enemy type — the Snappers — heavily armoured but slower than regular biters, with custom sprites and animations. Under K2's reworked combat, where bullets are physical projectiles that can miss, an armoured enemy that soaks the early weaponry forces the engineer to respect defence instead of kiting biters in a circle. The author describes the mod as compatible with most other mods, and it is on K2's official companion list.

Notes: By CybranM; 217K downloads. Krastorio 2's officially recommended combat companion. Power-spike verdict: passes — it makes combat harder and requires more defence investment, it does not raise any of the player's numbers.

The Krastorio Prodigy: Graphics

The three visual mods in this section make the K2 era **legible**. Two of them are Krastorio 2's own ecosystem: one is the K2 team's recommended progress tracker, and one makes Space Age's science packs carry K2-style tech-card icons so the whole research line matches the wave's signature mechanic. The third is a tiny clarity tweak for K2's roboports. None of them changes a game value; each one is a small investment that pays back across a very long wave.

Milestones

Version/Updated: Updated 23 Jun 2026 (page, 2.1-line v1.5.0); 2.0-line build v1.4.7 (24 Nov 2025), factorio_version 2.0.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Milestones' matches the API title exactly.

Dependencies: flib (a library the built-in mod manager installs automatically). No Krastorio 2 dependency is declared, but the Krastorio 2 page lists Milestones under its 'Recommended companion mods' — the K2 team's own compatibility confirmation.

Impact: Tracks the factory's key milestones — the first time you craft each important item, and how fast you reached it — and keeps a history you can look back on. K2's progress ladder is far longer than the base game's, so Milestones turns the whole wave into a scored, self-aware journey: the Prodigy knows exactly when the intergalactic transceiver became reachable, and how long the deep-science push took.

Notes: By Wiwiweb; 334K downloads. Krastorio 2's officially recommended progress companion. Pure UI and record-keeping; power-spike verdict: passes untouched.

K2 Tech Card Icons for Space Age

Version/Updated: Updated 1 Jul 2025 (page); 2.0-line build v0.1.2 (1 Jul 2025), factorio_version 2.0 — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'K2 Tech Card Icons for Space Age' matches the API title exactly.

Dependencies: Requires Space Age (built-in dependency) and Krastorio 2 Spaced Out (declared dependency on the portal). Optional: Maraxsis.

Impact: Gives Space Age's science packs Krastorio-2-style tech card icons, so the Space Age research line visually matches the K2 tech cards that this wave's research overhaul is built around. In a K2SO run the lab screen stays visually coherent from the first tech card to the singularity.

Notes: By jwinters2; 290 downloads. Small adoption but verified, active, and dependent on the K2SO bridge itself — which makes it provably K2SO-compatible. Pure icon reskin; no recipe, icon size or balance value changes. Power-spike verdict: passes untouched.

Krastorio2 - roboport mode colored textures

Version/Updated: Updated 24 Jun 2025 (page); 2.0-line build v2.0.4 (24 Jun 2025), factorio_version 2.0 — the latest release on any line.

Verified: YES — portal page + API fetched 2026-08-03; page title 'Krastorio2 - roboport mode colored textures' matches the API title exactly.

Dependencies: None required — the mod declares Krastorio 2 and Krastorio 2 Spaced Out as optional dependencies, so its textures light up automatically when either anchor is present.

Impact: Tints the rings of Krastorio 2's roboports normal, green or orange based on which mode each roboport is set to, so the state of the robot network — connected, disconnected, passive — is visible across the factory at a glance. A small clarity win on the logistics layer of a very long wave.

Notes: By Quezler; 1.27K downloads. Pure visual; no roboport behaviour, mode or balance value is touched. Power-spike verdict: passes untouched.

Glossary

Assembling machine — a machine that crafts a chosen recipe automatically while inserters keep it fed; assembling machine 1 handles simple two-ingredient recipes, and higher tiers add speed, ingredients and module slots.

Inserter — a moving arm that picks one item from behind it and places it in front, connecting belts, chests and machines into a working loop.

Transport belt — a one-way conveyor that carries items at a fixed speed (a yellow belt moves 15 items per second); underground belts dive under obstacles and splitters divide one belt into two.

Burner machine — a machine (mining drill, furnace, inserter, boiler) that runs on fuel it holds itself rather than electricity; they are self-contained but stop the moment their fuel runs out.

Steam power — the early electric chain: an offshore pump on a lake shore supplies up to 200 boilers' worth of water (1,200 units per second), each boiler turns water into 165°C steam (60 per second, one coal roughly every 2.2 seconds), and each steam engine consumes 30 steam per second to make 900 kW — the modern 2.0 ratio is 1 pump : 200 boilers : 400 engines, two engines per boiler.

Alt-mode — the toggle (Alt key) that overlays every entity with its recipe, contents or direction, making the whole factory readable at a glance.

Science pack — the research currency consumed by labs; red (automation) science is copper plates + iron gear wheels, green (logistic) science is a transport belt + an inserter, and later packs add oil and advanced chains.

Research — a technology unlocked by feeding science packs into labs; the research queue lets you line up several technologies so labs never idle.

Pollution — the red cloud emitted by machines that spreads across the map; when it reaches a biter nest, the nest musters an attack, so a factory's noise determines its trouble.

Biter / spawner & evolution factor — the native life of Nauvis; spawners absorb pollution to launch attack waves, and biters are the soldiers that follow. The evolution factor is the world's hidden difficulty meter — raised slowly by time, faster by pollution, and most of all by destroying biter nests — and together the two decide how fast and how tough the attacks get.

Blueprint — a saved copy of a group of buildings (the B key opens the library) that you can stamp down anywhere, letting a whole production block be recreated in seconds.

Pipette — the Q-key tool that grabs the item you are hovering over, so you can build more of anything by looking at it.

Oil processing — the oil chain that opens with chemical science: a pumpjack on a crude-oil well extracts it (output declines as the well depletes), an oil refinery turns crude into petroleum gas, light oil and heavy oil in one step (stopping when any output tank is full), and cracking converts heavier fractions into lighter ones (heavy to light, light to petroleum) to unblock a jammed line.

Rail signal / chain signal — track-side signals that divide rails into blocks so two trains cannot collide; chain signals read ahead and guard crossings.

Locomotive / cargo wagon — the engine and the cargo of a train; a schedule (go to stop A until full, stop B until empty) makes a hauling line run forever.

Rocket — the endgame capstone: a rocket silo assembles rocket parts and launches the rocket, and its research needs every science pack except military. Since 2.0.7 each part costs 10 low density structures, 10 processing units and 10 rocket fuel, and a full rocket needs 100 of them (1,000 of each). The classic payload is a satellite, which returns 1,000 space science packs to the cargo landing pad about 29 seconds after launch.

Main bus — a city-block-era layout idea (full detail in Wave 1) where a few broad parallel belts carry the core materials past every production block, pulled off by splitters.

Circuit network — the wire system that carries signal values between entities; red and green wires form separate networks, and any device wired to a network can read the sum of its signals.

Combinator — a circuit-network device that computes: the constant combinator broadcasts up to 20 fixed values, the arithmetic combinator calculates a result, and the decider combinator passes a signal through only when a condition is true.

Logistic network — the robot delivery system; roboports whose orange zones touch join into one network, and the network's robots move items between its provider, requester, storage and buffer chests.

Roboport — the charging bay and network anchor for construction and logistic robots; it creates a 50x50-tile logistic zone and a 110x110-tile construction zone.

Logistic robot — a flying robot that carries items between chests inside a logistic network, one item at a time by default (up to four with research), flying at about 3 tiles per second.

Logistic chests — the chest family that plugs a factory into the logistic network: a requester chest asks the network to deliver a chosen item up to a set amount, a passive provider chest makes its contents available without pushing them out, a storage chest holds surplus and deconstruction drops, and a buffer chest both requests and provides, staging materials near a consumer.

Module — an expensive, research-gated item slotted into machines and beacons, in four types: speed modules raise crafting speed (+20%, +30%, +50% by tier) while raising power and pollution; efficiency modules cut energy consumption and pollution (−30%, −40%, −50% by tier); productivity modules give a chance of free bonus output (+4%, +6%, +10% by tier) while slowing the machine and raising its power use; and quality modules give a chance (+1%, +2%, +2.5% by tier) of crafting an item one quality tier higher.

Quality — the five-tier item system (normal, uncommon, rare, epic, legendary); higher-tier items are stronger, and quality rolls on crafted items improve with quality modules.

Recycler — a building that breaks crafted items back into a quarter of their ingredients, rolling quality again — the gate that makes quality upcycling loops work.

Beacon — a building that broadcasts its modules' effects to every compatible machine in a 9x9 area around it, with diminishing returns as more beacons target the same machine.

Train limit — a number set on a train stop capping how many trains may head for it at once, the first and best anti-deadlock habit on a shared network.

Belt stacking — the Wave 1 mechanic (Deadlock's Stacking Beltboxes) that combines several copies of an item into one stacked belt item, multiplying what a belt lane can carry.

Krastorio 2 — the Wave 2 overhaul anchor: rebalances nearly every recipe, adds over 40 buildings, over 100 technologies and three new resources, replaces science packs with tech cards, and reworks combat so bullets are physical projectiles.

Krastorio 2 Spaced Out — the Wave 2 bridge mod that merges Krastorio 2 with Space Age; K2 requires it to run correctly when Space Age is enabled ("the game will load, but progression will be broken" without it).

Imersite — Krastorio 2's rare end-game resource: "useless on its own, but when used in advanced manufacturing processes, exhibits ludicrous tensile strength and superconductive abilities"; the material the end-game machines are built from.

Mineral water — Krastorio 2's second water: normal water "impregnated with various silts and dusts", used primarily in the production of lithium; a separate fluid supply line from ordinary water.

Rare metals — Krastorio 2's mined metal resource, "an analogous representation of various rare elements (i.e. titanium, platinum, gold)"; feeds the quality tier of the metal economy.

Tech card — Krastorio 2's replacement for science packs: produced in assembling machines and inserted into research computers; there are three tiers, each requiring its own laboratory, and early tiers are phased out as the game progresses.

Research computer — Krastorio 2's laboratory: the building tech cards are inserted into, replacing the vanilla lab.

Advanced tech card — a K2SO research tier that unlocks powerful new buildings on the Space Age planets (advanced furnace on Vulcanus, laser artillery on Fulgora, advanced chemical lab on Gleba, and more).

Matter tech card — a K2SO research tier made from samples of every resource that enables material transmutation, including of Space Age resources.

Singularity tech card — a K2SO space-exclusive research tier made of pure matter and asteroid chunks that unlocks instant interplanetary travel and the highest tier of logistics.

Intergalactic transceiver — Krastorio 2's end-game win building; under K2SO it must be built in space.

Mod portal — the official Factorio website (mods.factorio.com) where every mod in this guide is hosted; each mod's page shows its description, downloads, version history and dependencies, and it is the only source the guide installs from.

factorio_version — the game version a mod's release is built against, shown on its portal page; this guide targets Factorio 2.0.77, so every chosen mod is a 2.0-line build, and newer 2.1-line builds are deliberately out of scope.

Mod dependency — another mod a given mod needs to run, declared on its portal page; required dependencies install automatically with the mod, optional ones are listed for you to choose, and the mod manager refuses to enable a mod whose dependencies are missing.

Mod conflict — two mods that change the same recipes, buildings or data and therefore cannot run together; the mod manager blocks the conflicting pair, and the fix is usually a bridge mod (like Krastorio 2 Spaced Out for Krastorio 2 + Space Age) that merges them.

mods/ folder — the folder in the game's data directory where mods live as .zip files; the built-in mod manager reads and writes it automatically, so you normally never touch it, though it is the place to check what is actually installed.

Built-in mod manager — the game's own mod browser (main menu -> Mods) that searches the portal, installs, updates, and enables or disables mods and their dependencies; it is the only mod tool this guide uses, so no third-party organizer is needed.

Load order ("later wins") — the rule that when two mods change the same thing, the one loaded later in the load order wins; Factorio sorts mods by their dependencies automatically and you adjust the rest in the mod manager, so disagreements between mods are settled before the game world loads.